

BDA Course 2026

Lecture 4

Outline Chapter 10

- 10.1 Numerical integration (overview)
- 10.2 Distributional approximations (overview, more in Chapter 4 and 13)
- 10.3 Direct simulation and rejection sampling (overview)
- 10.4 Importance sampling
 - used in PSIS-LOO (Lecture 9) and prior sensitivity analysis
- 10.5 How many simulation draws are needed?
 - see chapter notes for how many significant digits to report
 - this week focus on independent draws and importance sampling, next week necessary adjustments needed for Markov chain Monte Carlo
- 10.6 Software (can be skipped)
- 10.7 Debugging (can be skipped)

Expectations everywhere

- The posterior is the central object of Bayesian inference.

Expectations everywhere

- The posterior is the central object of Bayesian inference.
- It summarizes all the information about the parameters given the data.

Expectations everywhere

- The posterior is the central object of Bayesian inference.
- It summarizes all the information about the parameters given the data.
- But often we most directly care about other quantities derived from the posterior

Expectations everywhere

- The posterior is the central object of Bayesian inference.
- It summarizes all the information about the parameters given the data.
- But often we most directly care about other quantities derived from the posterior
- Many of them can be expressed as expectations of functions of the parameters:

$$E_{p(\theta|y)}[h(\theta)] = \int h(\theta) p(\theta|y) d\theta, \quad \text{where} \quad p(\theta|y) = \frac{p(y|\theta)p(\theta)}{\int p(y|\theta)p(\theta) d\theta}$$

- Posterior mean: $h(\theta) = \theta$ $E[\theta]$
- Posterior variance: $h(\theta) = (\theta - E[\theta])^2$ $\text{Var}(\theta)$
- Posterior probability of an event: $h(\theta) = \mathbb{1}(\theta \in A)$ $P(\theta \in A \mid y)$
- Posterior predictive: $h(\theta) = p(\tilde{y} \mid \theta)$ $p(\tilde{y} \mid y)$

The hurdle: normalizing constants

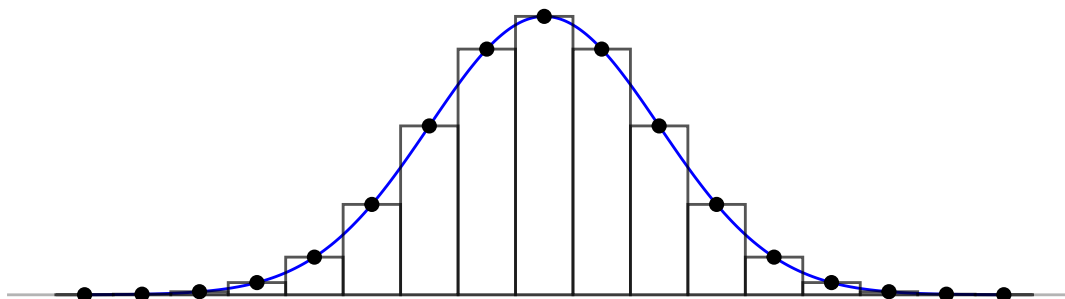
- Often $p(y|\theta)p(\theta)$ is easy to evaluate, but the normalizing integral is not.
 - We work with $q(\theta|y) = p(y|\theta)p(\theta) \propto p(\theta|y)$ instead.

The hurdle: normalizing constants

- Often $p(y|\theta)p(\theta)$ is easy to evaluate, but the normalizing integral is not.
 - We work with $q(\theta|y) = p(y|\theta)p(\theta) \propto p(\theta|y)$ instead.
- Methods to compute the posterior:
 - Conjugate priors and analytic solutions (Ch 1-5, Lec 2-3)
 - Grid approximation and other quadrature rules (Ch 3, 10, Lec 3-4)
 - Monte Carlo, rejection sampling and importance sampling (Ch 10, Lec 4)
 - Markov Chain Monte Carlo (Ch 11-12, Lec 5-6)
 - Distributional approximations (Laplace, VB, EP) (Ch 4, 13)

Quadrature integration

- The simplest quadrature integration is grid integration

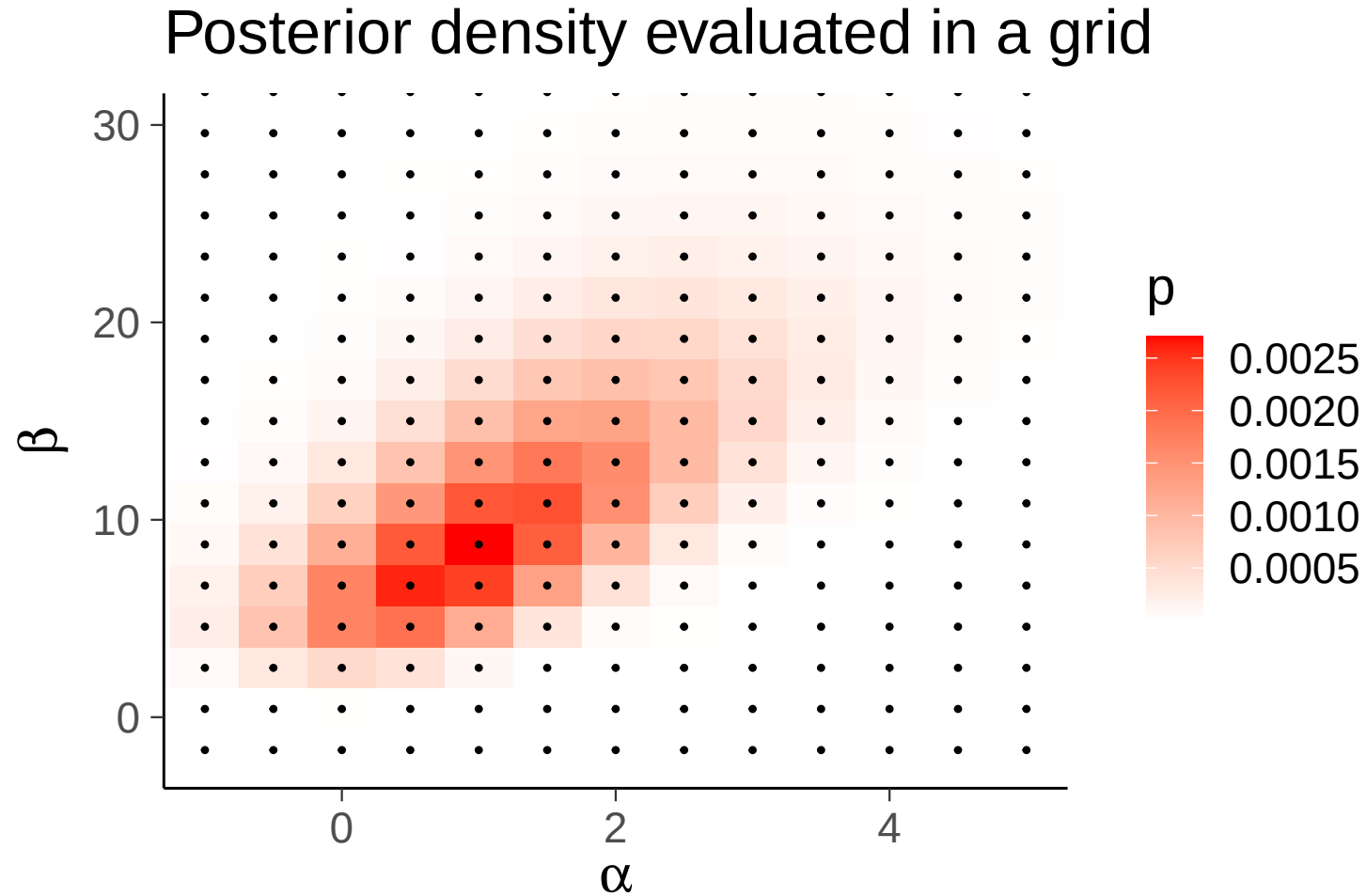


$$E[\theta] \approx \sum_{t=1}^T \theta^t w^t,$$

where w^t is the normalized probability of a grid cell t , and θ^t are the center of the grid cells

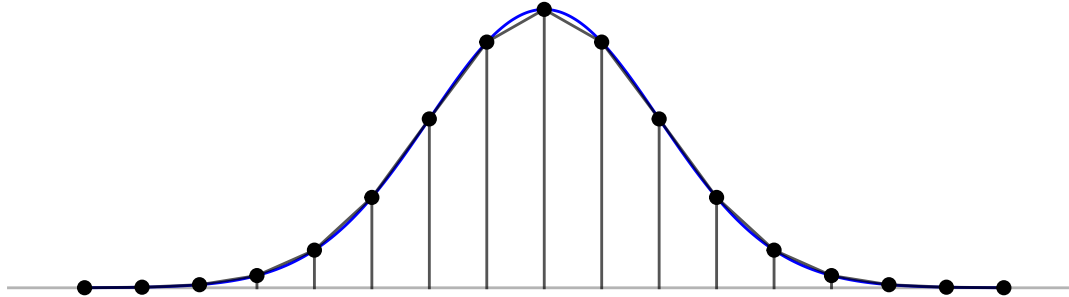
Bioassay - grid approximation

- We already saw in one example in 2D for the bioassay problem.



Better grid methods

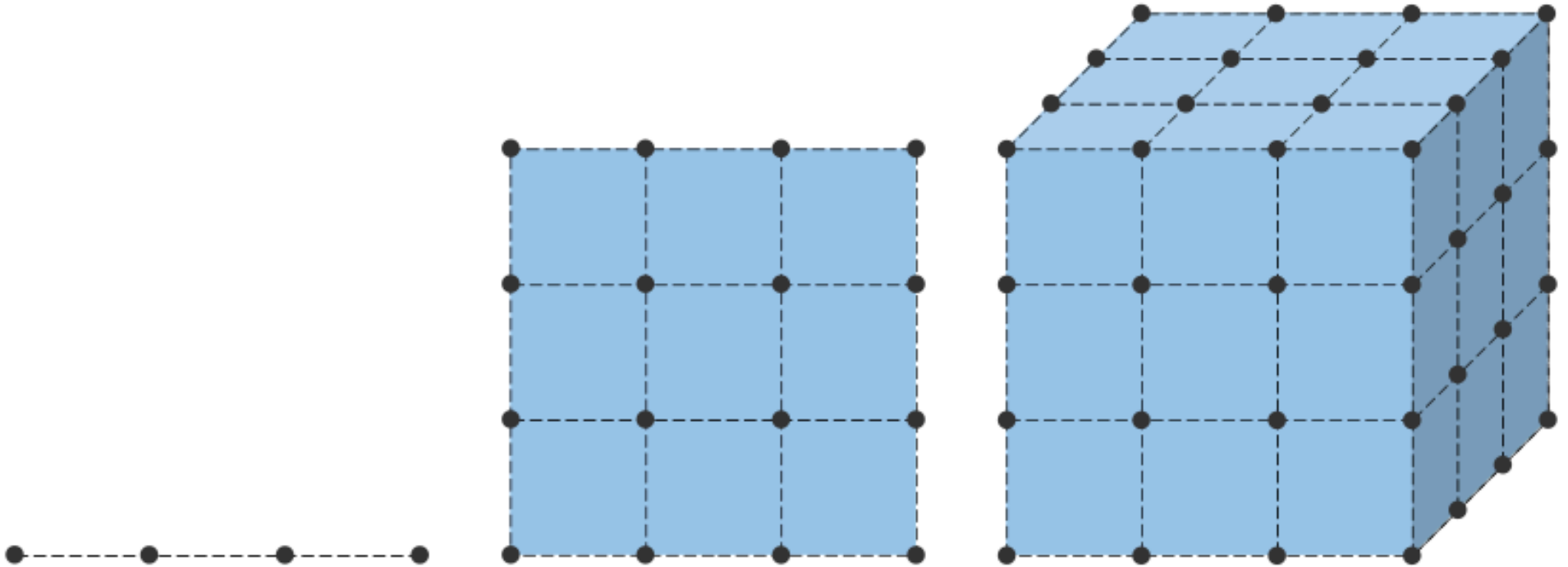
- In 1D further variations with better accuracy, e.g. trapezoid



- Adaptive quadrature methods add evaluation points where needed , e.g., R function `integrate()`
- In 2D and higher there are more sophisticated quadrature methods available, like nested quadrature and product rules

Grid sampling and curse of dimensionality

- In general the number of evaluations increase exponentially c^D



Grid sampling and curse of dimensionality

- In general the number of evaluations increase exponentially c^D

Grid sampling and curse of dimensionality

- In general the number of evaluations increase exponentially c^D
- e.g. 50 or 1000 grid points per dimension, and 10 dimensions
 - $\rightarrow 50^{10} \approx 1\text{e}17$ grid points
 - $\rightarrow 1000^{10} \approx 1\text{e}30$ grid points

Grid sampling and curse of dimensionality

- In general the number of evaluations increase exponentially c^D
- e.g. 50 or 1000 grid points per dimension, and 10 dimensions
 - $\rightarrow 50^{10} \approx 1\text{e}17$ grid points
 - $\rightarrow 1000^{10} \approx 1\text{e}30$ grid points
- On a standard laptop we can evaluate the density of normal distribution about 50 million times per second
 - $\rightarrow$ evaluation in $1\text{e}17$ grid points would take 60 years
 - $\rightarrow$ evaluation in $1\text{e}30$ grid points would take 600 billion years

Grid sampling and curse of dimensionality

- To make things worse
 - We often don't know beforehand where the posterior mass is
 - need to choose wider enough box
 - Easy to get a lot of evaluations in regions with negligible posterior mass
 - In higher dimensions this problem becomes even more severe

What if we abandon deterministic?

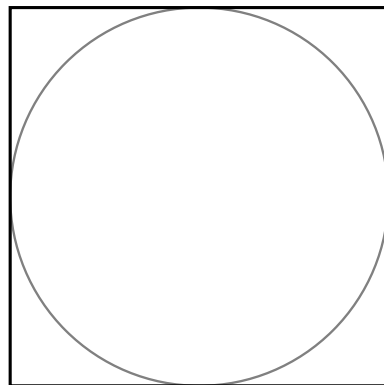
- Instead of using a deterministic rule (like a grid), sample points stochastically
- Intuition: by sampling stochastically, we may explore the space more efficiently, especially in high dimensions
- Good deterministic methods require fewer function evaluations, but they may not be applicable

Monte Carlo methods

- A broad class of algorithms that use repeated random sampling to compute numerical results
- The underlying idea: some problems may be very difficult (or impossible) to tackle exactly
- Conceptualized by the Polish mathematician Stanisław Ulam
 - The idea was extended by others like John von Neumann and Nicholas Metropolis, and many more.
 - Early computers era
 - Used already before computers (at least from 18th century onwards)
- Also called Monte Carlo experiments or Monte Carlo simulations
- Markov Chain Monte Carlo (MCMC) methods are a subclass (next week)

Monte Carlo: Computing π

- Sample points uniformly in the square $[-1, 1]^2$
- Count the number of points that fall inside the circle



- The probability a point lands inside the circle:

$$P(\text{inside circle}) = \frac{\text{area of circle}}{\text{area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}$$

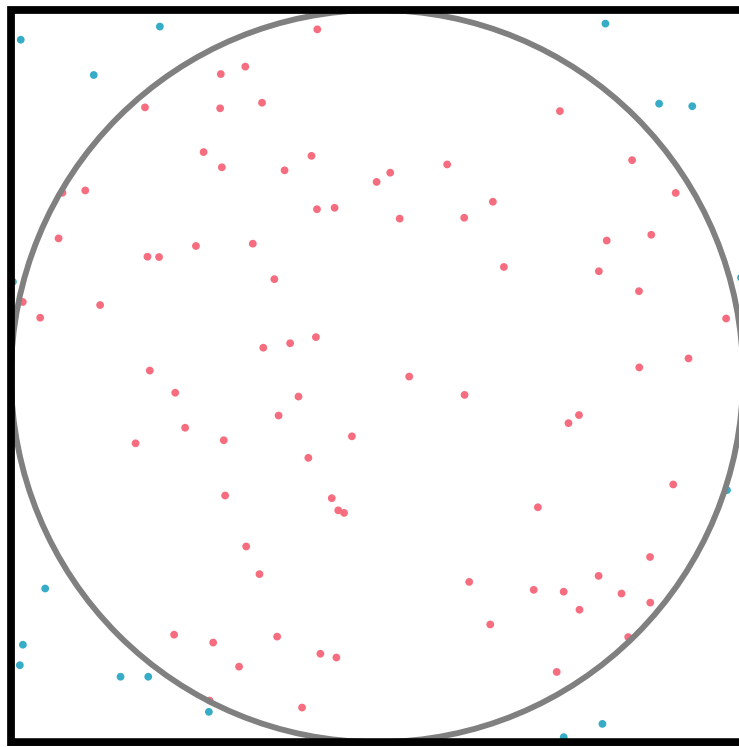
- And from that we can estimate π :

$$\hat{\pi} = 4 \times \frac{\text{points inside circle}}{\text{total points}}$$

Monte Carlo: Computing π

- Estimate π by sampling points uniformly in the square $[-1, 1]^2$
- Compute the fraction that fall inside the unit circle

$$\hat{\pi} = 4 \times \frac{\text{points inside circle}}{\text{total points}}$$



draws = 100

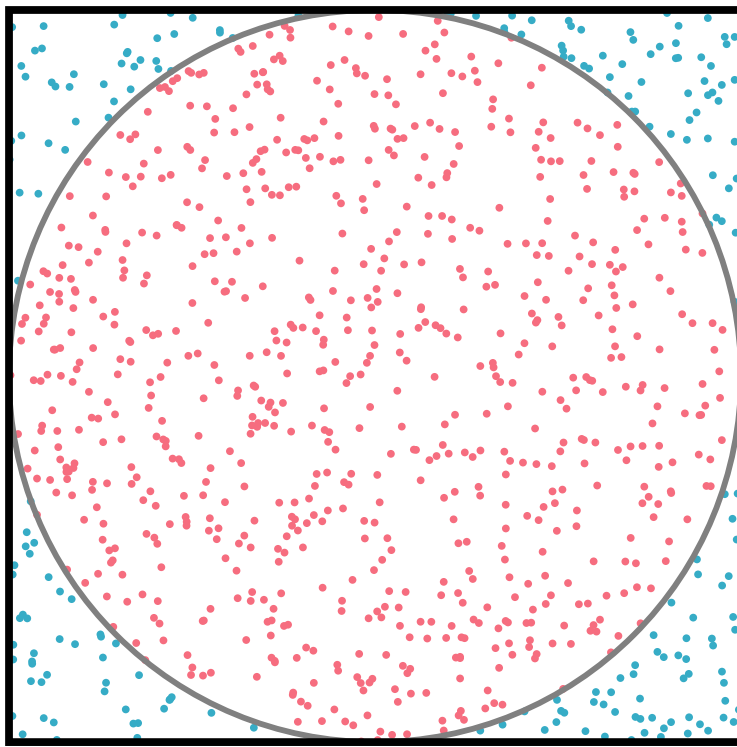
$\hat{\pi} = 3.320$

error = 5.374%

Monte Carlo: Computing π

- Estimate π by sampling points uniformly in the square $[-1, 1]^2$
- Compute the fraction that fall inside the unit circle

$$\hat{\pi} = 4 \times \frac{\text{points inside circle}}{\text{total points}}$$



draws = 1000

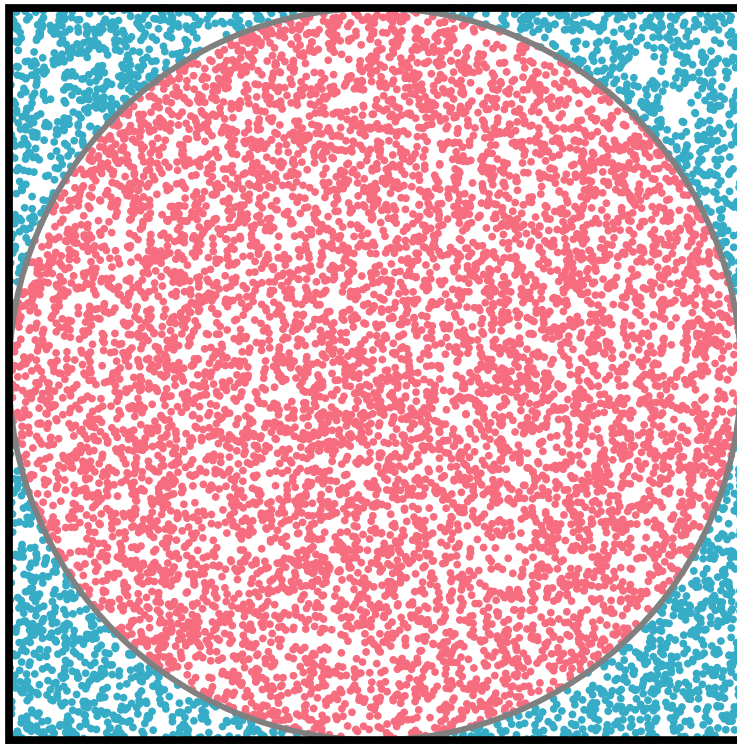
$\hat{\pi} = 3.164$

error = 0.708%

Monte Carlo: Computing π

- Estimate π by sampling points uniformly in the square $[-1, 1]^2$
- Compute the fraction that fall inside the unit circle

$$\hat{\pi} = 4 \times \frac{\text{points inside circle}}{\text{total points}}$$



draws = 10000

$\hat{\pi} = 3.142$

error = 0.013%

Monte Carlo: Computing π

- There are better methods to estimate π
- But this illustrates the basic idea of Monte Carlo sampling.
- As $n \rightarrow \infty$, $\hat{\pi} \rightarrow \pi$
- The variance of the estimate also decreases (more on this later)

Monte Carlo

- In the π example, we simulated from a uniform distribution and discarded samples that fell outside the circle
 - We will come back to this idea later (rejection sampling)

Monte Carlo

- In the π example, we simulated from a uniform distribution and discarded samples that fell outside the circle
 - We will come back to this idea later (rejection sampling)
- Now we focus on direct sampling:
 - Simulate draws from the target distribution (like the posterior)
 - Use these draws to compute what we want
 - Means, deviations, quantiles
 - Histograms
 - etc.

Monte Carlo

- In the π example, we simulated from a uniform distribution and discarded samples that fell outside the circle
 - We will come back to this idea later (rejection sampling)
- Now we focus on direct sampling:
 - Simulate draws from the target distribution (like the posterior)
 - Use these draws to compute what we want
 - Means, deviations, quantiles
 - Histograms
 - etc.
- A collection of draws is a sample

Monte Carlo: Estimating Posterior Quantities

Posterior mean

$$\mathbb{E}[\theta \mid y] = \int \theta p(\theta \mid y) \, d\theta$$

$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n \theta^{(i)}, \quad \theta^{(i)} \sim p(\theta \mid y)$$

Posterior variance

$$\text{Var}[\theta \mid y] = \int (\theta - \mathbb{E}[\theta \mid y])^2 p(\theta \mid y) \, d\theta$$

$$\widehat{\text{Var}} = \frac{1}{n} \sum_{i=1}^n (\theta^{(i)} - \hat{\theta})^2$$

Monte Carlo: Estimating Posterior Quantities

90% equal-tailed credible interval

- Sort into order statistics: $\theta_{(1)} \leq \theta_{(2)} \leq \dots \leq \theta_{(n)}$
- The p -th percentile sits (approximately) at rank $k = p(n + 1)$
- 90% CI: $[\theta_{(0.05(n+1))}, \theta_{(0.95(n+1))}]$
- In practice we can use functions like:
 - `quantile(theta, probs = c(0.05, 0.95))` (R)
 - `np.quantile(theta, [0.05, 0.95])` (Python).

Do I have enough simulated draws?

- If draws are independent
 - usual methods to estimate the uncertainty due to a finite number of observations
- Correlated draws
 - Require additional work
 - We will discuss this in the context of Markov chain Monte Carlo next week.

The Central Limit Theorem (CLT)

- For an expectation of unknown quantity $E(\theta) \approx \frac{1}{S} \sum_{s=1}^S \theta^s$

The Central Limit Theorem (CLT)

- For an expectation of unknown quantity $E(\theta) \approx \frac{1}{S} \sum_{s=1}^S \theta^s$
- CLT: The distribution of the $E(\theta)$ approaches normal distribution with variance $\frac{\sigma_\theta^2}{S}$

The Central Limit Theorem (CLT)

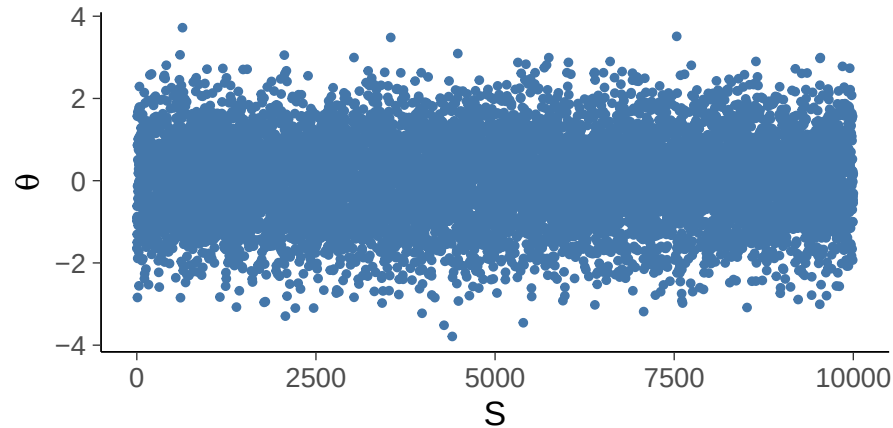
- For an expectation of unknown quantity $E(\theta) \approx \frac{1}{S} \sum_{s=1}^S \theta^s$
- CLT: The distribution of the $E(\theta)$ approaches normal distribution with variance $\frac{\sigma_\theta^2}{S}$
- CLT applies if:
 - S is *big enough*,
 - θ^s are independent,
 - $p(\theta)$ has finite variance,

The Central Limit Theorem (CLT)

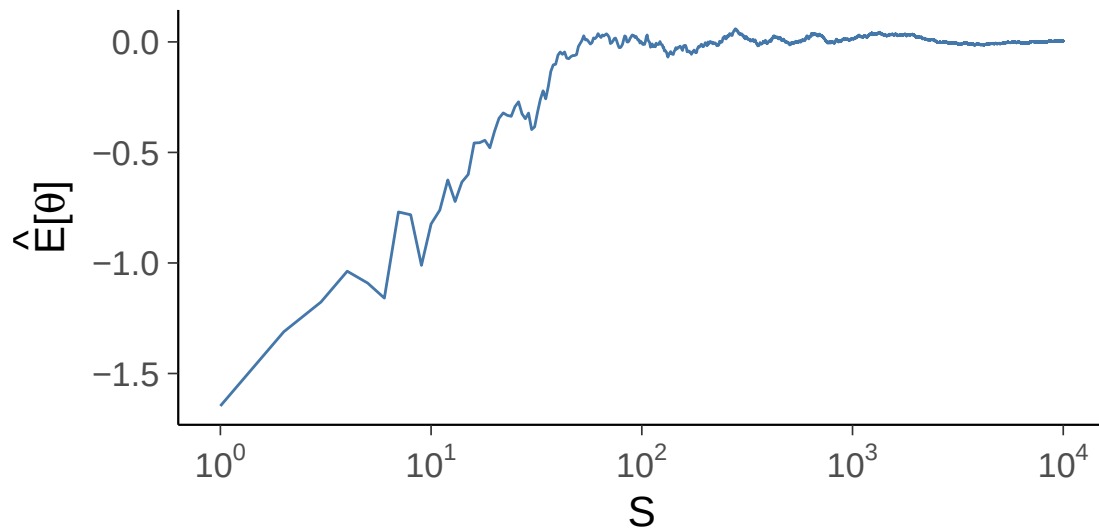
- For an expectation of unknown quantity $E(\theta) \approx \frac{1}{S} \sum_{s=1}^S \theta^s$
- CLT: The distribution of the $E(\theta)$ approaches normal distribution with variance $\frac{\sigma_\theta^2}{S}$
- CLT applies if:
 - S is *big enough*,
 - θ^s are independent,
 - $p(\theta)$ has finite variance,
- See BDA3 Ch 4

CLT, mean of Gaussian draws

- Suppose we have S i.i.d. draws $\theta^{(s)} \sim N(0, 1)$ for $s = 1, \dots, S$

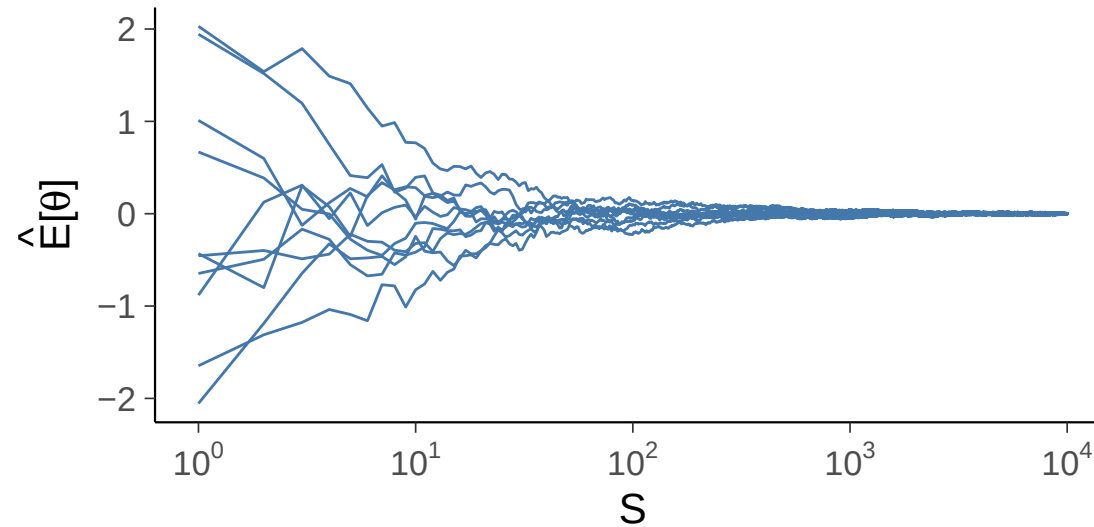


- Now let see how the mean behaves as we accumulate draws from 1 to S



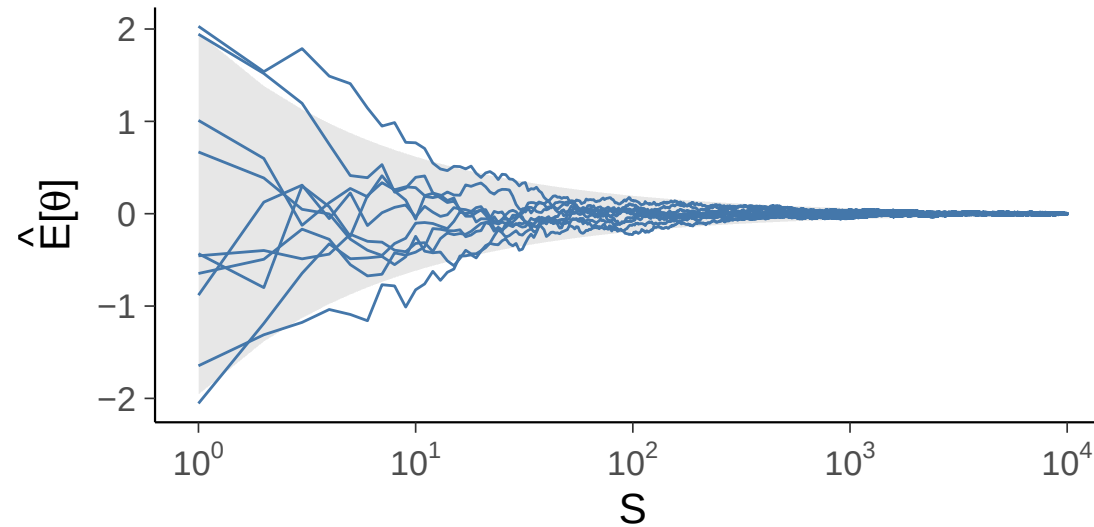
CLT, variability across repeated samples

- We now repeat for a few set of samples, each of size S



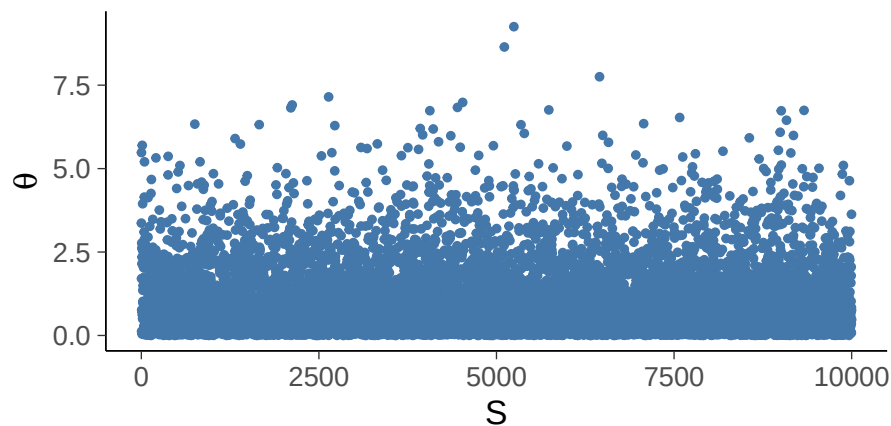
CLT, variability across repeated samples

- We now repeat for a few set of samples, each of size S

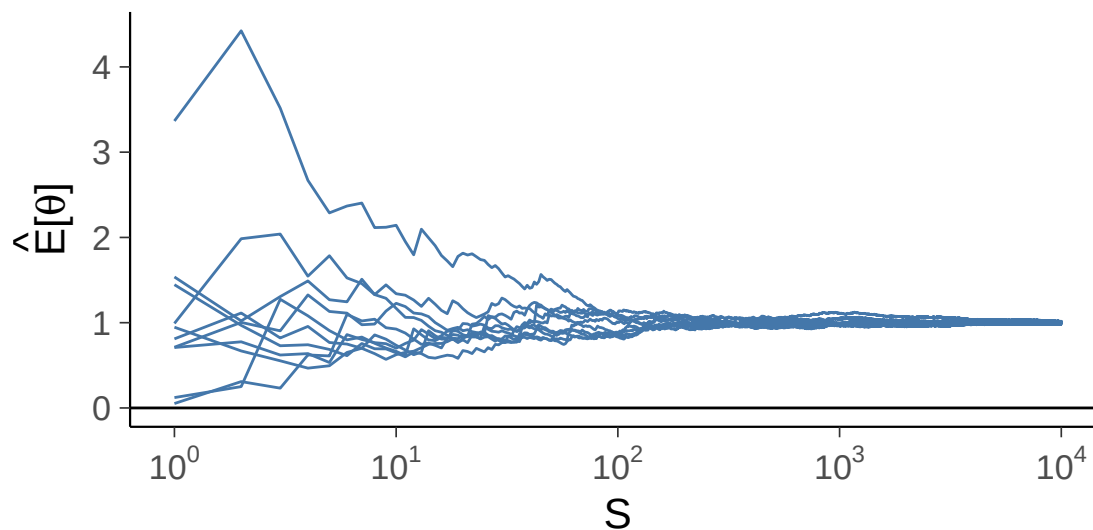


CLT, mean of Exponential draws

- Suppose we have S i.i.d. draws $\theta^{(s)} \sim \text{Exp}(1)$ for $s = 1, \dots, S$

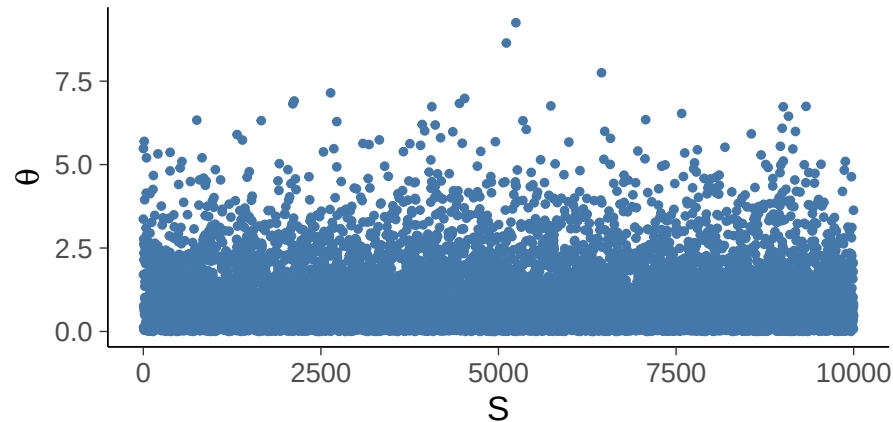


- Now let see how the mean behaves as we accumulate draws from 1 to S

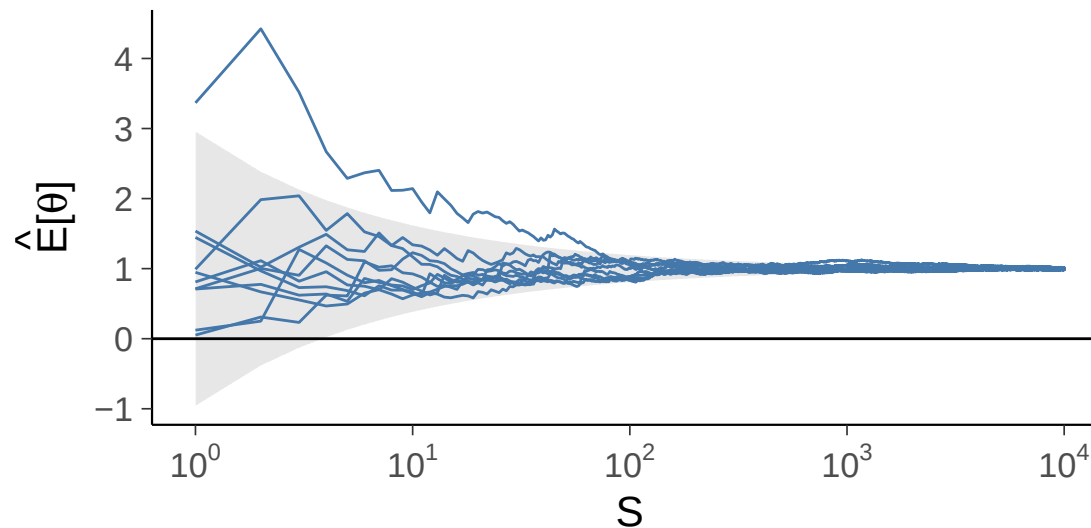


CLT, mean of Exponential draws

- Suppose we have S i.i.d. draws $\theta^{(s)} \sim \text{Exp}(1)$ for $s = 1, \dots, S$

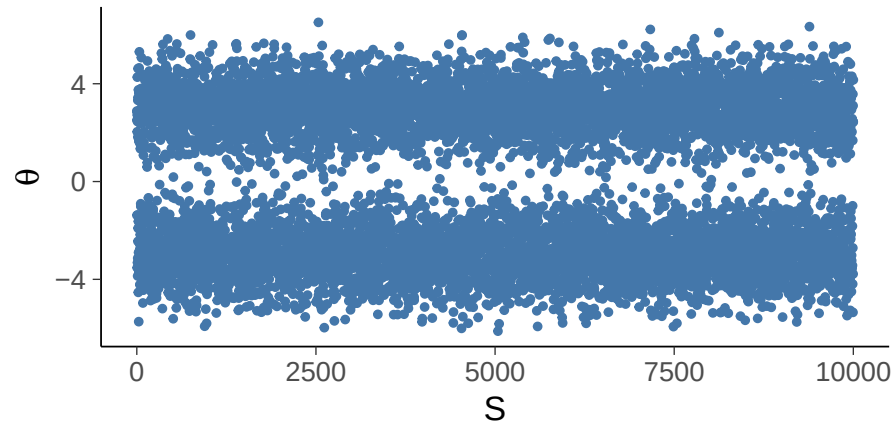


- Now let see how the mean behaves as we accumulate draws from 1 to S

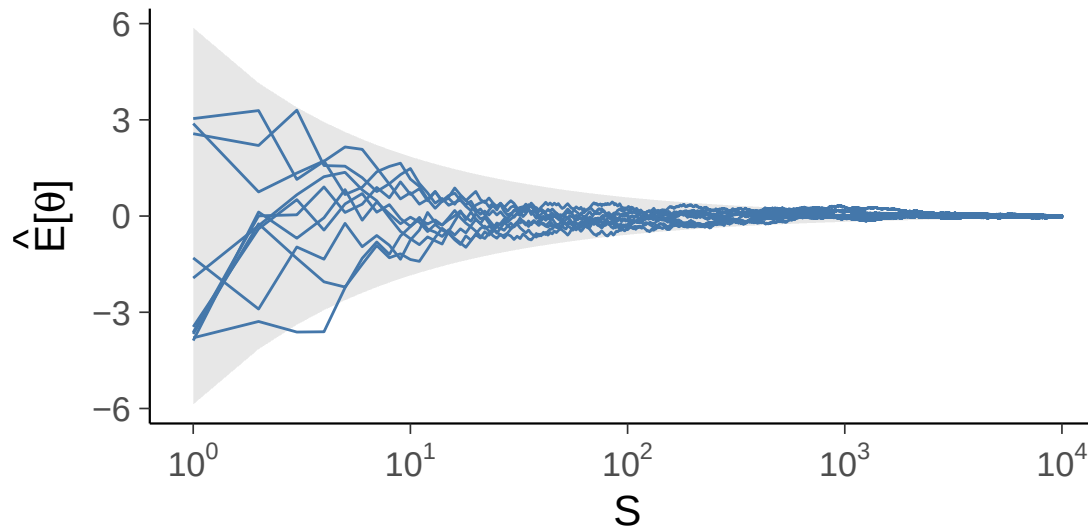


CLT, mean of a bimodal distribution

- Suppose we have S i.i.d. draws $\theta^{(s)} \sim 0.5N(-3, 1) + 0.5N(3, 1)$ for $s = 1, \dots, S$

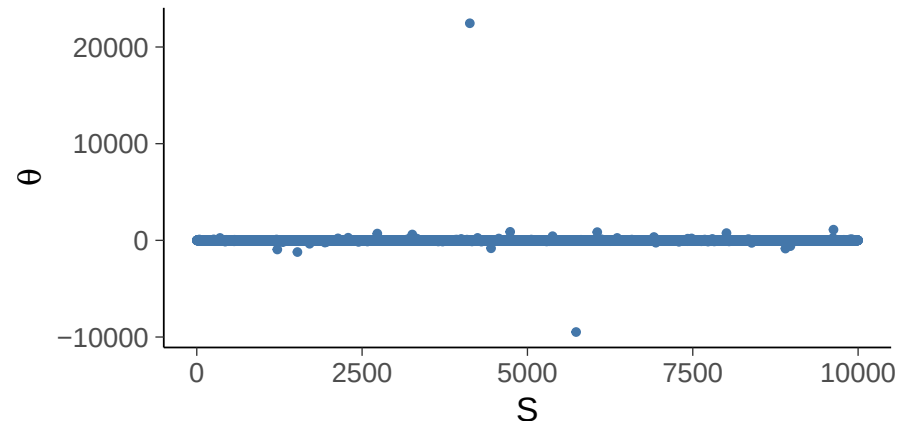


- Now let see how the mean behaves as we accumulate draws from 1 to S

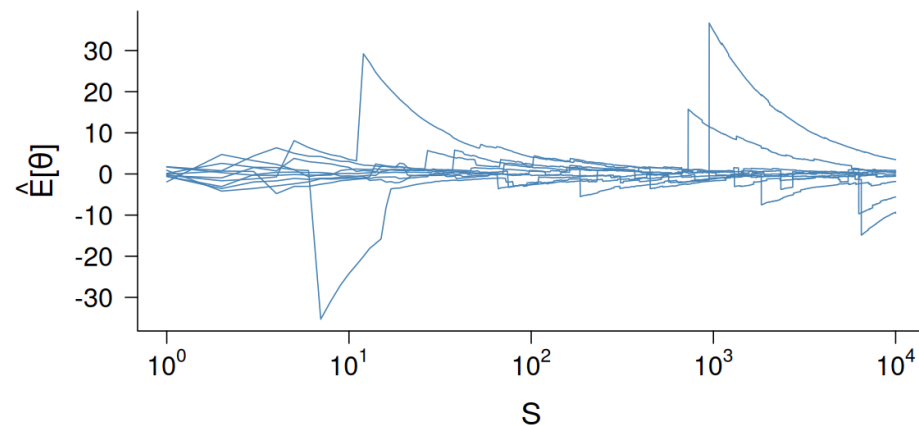


CLT not always applies: Cauchy draws

- Suppose we have S i.i.d. draws $\theta^{(s)} \sim t_1$ for $s = 1, \dots, S$
- Mean is undefined and variance is infinite



- Now let's see how the mean behaves as we accumulate draws from 1 to S



CLT and Monte Carlo standard error (MCSE)

- Expectation of unknown quantity $E(\theta) \approx \frac{1}{S} \sum_{s=1}^S \theta^s$
 - If S is big,
 - θ^s are independent,
 - $p(\theta)$ has finite variance,

CLT and Monte Carlo standard error (MCSE)

- Expectation of unknown quantity $E(\theta) \approx \frac{1}{S} \sum_{s=1}^S \theta^s$
 - If S is big,
 - θ^s are independent,
 - $p(\theta)$ has finite variance,
- Then central limit theorem (CLT) applies
 - The distribution of the $E(\theta)$ approaches normal distribution with variance $\frac{\sigma_\theta^2}{S}$
 - this variance is independent on dimensionality of θ
 - $\frac{\sigma_\theta}{\sqrt{S}}$ is called Monte Carlo standard error (MCSE)
 - In practice, σ_θ will be estimated by $\sqrt{\frac{1}{S-1} \sum_{s=1}^S (\theta^s - E(\theta))^2}$

Further resources on the CLT

- 3Blue1Brown YouTube videos with nice visualisations
 - CLT with discrete distributions: *But what is the Central Limit Theorem?* <https://www.youtube.com/watch?v=zeJD6dqJ5lo>
 - CLT with continuous distributions: *Convolutions / Why $X+Y$ in probability is a beautiful mess* <https://www.youtube.com/watch?v=laSGqQa5O-M>

Why compute MCSE at all?

- MCSE tells us the error in our Monte Carlo estimate

Why compute MCSE at all?

- MCSE tells us the error in our Monte Carlo estimate
- Computers usually return results with more digits than are actually meaningful

Why compute MCSE at all?

- MCSE tells us the error in our Monte Carlo estimate
- Computers usually return results with more digits than are actually meaningful
- If the MCSE is 0.01, it does not make sense to report 4.547 as the result

Why compute MCSE at all?

- MCSE tells us the error in our Monte Carlo estimate
- Computers usually return results with more digits than are actually meaningful
- If the MCSE is 0.01, it does not make sense to report 4.547 as the result
- MCSE should be small relative to posterior uncertainty

Why compute MCSE at all?

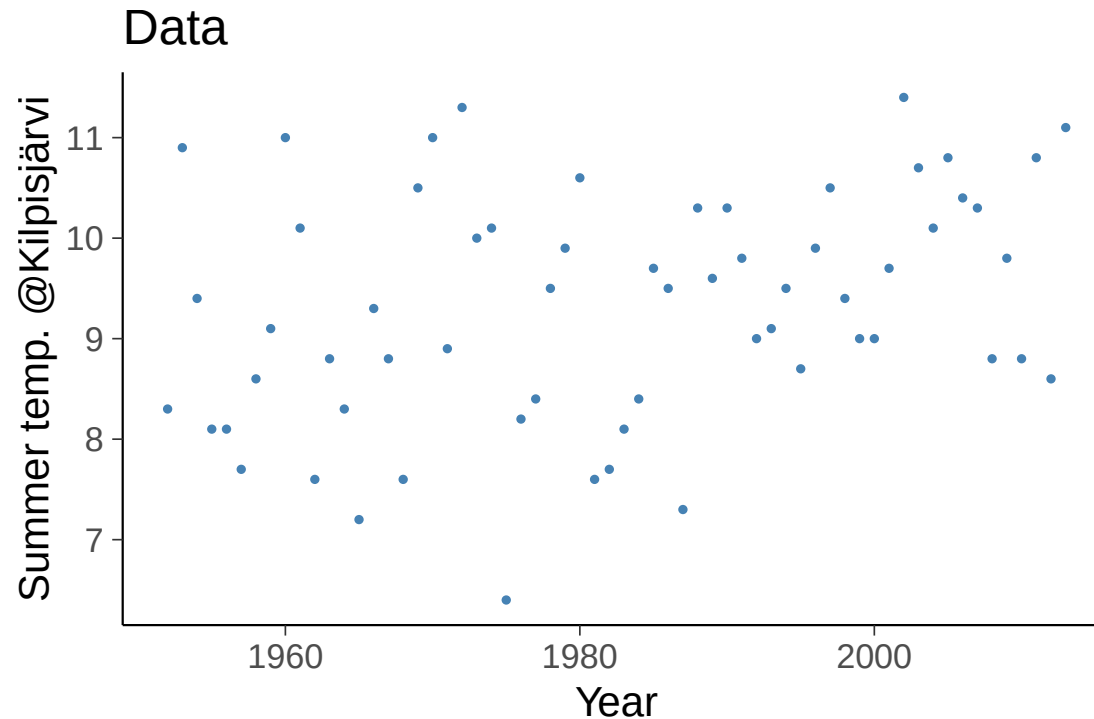
- MCSE tells us the error in our Monte Carlo estimate
- Computers usually return results with more digits than are actually meaningful
- If the MCSE is 0.01, it does not make sense to report 4.547 as the result
- MCSE should be small relative to posterior uncertainty
- MCSE is useful when deciding how to summarize and report results

Why compute MCSE at all?

- MCSE tells us the error in our Monte Carlo estimate
- Computers usually return results with more digits than are actually meaningful
- If the MCSE is 0.01, it does not make sense to report 4.547 as the result
- MCSE should be small relative to posterior uncertainty
- MCSE is useful when deciding how to summarize and report results
- Worth reading: [digits case study](#)

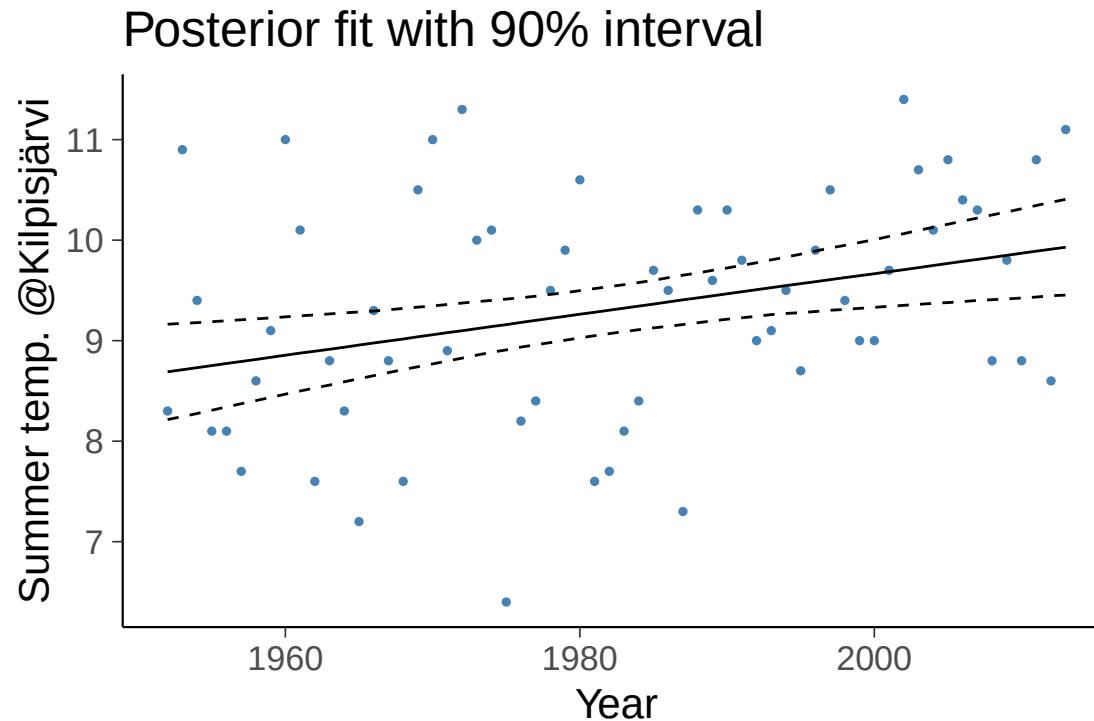
Example: Kilpisjärvi summer temperature

Average temperature in June, July, and August at Kilpisjärvi, Finland in 1952–2013



Example: Kilpisjärvi summer temperature

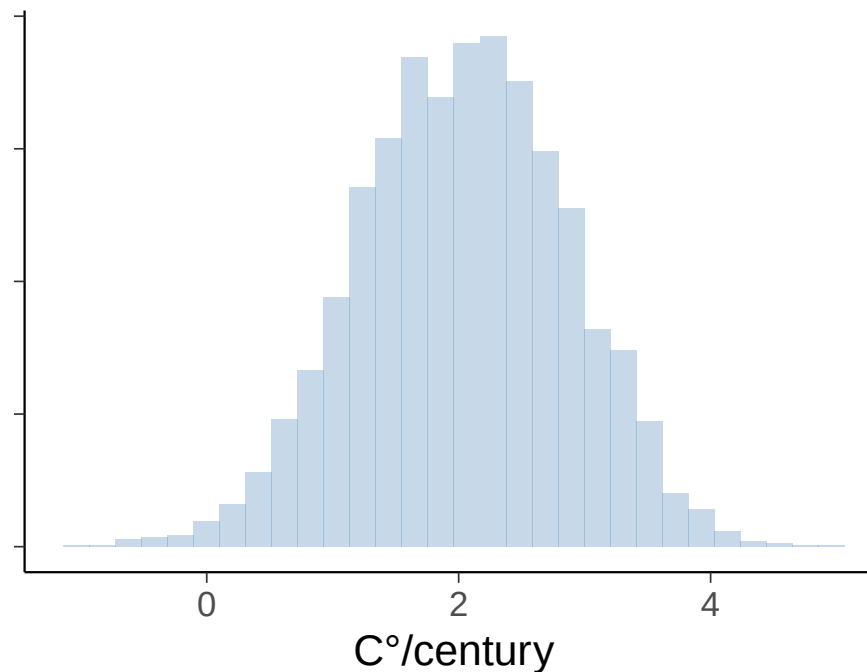
Average temperature in June, July, and August at Kilpisjärvi, Finland in 1952–2013



Example: Kilpisjärvi summer temperature

- The posterior mean is $\approx 1.9^\circ\text{C}$ per century
- The 90% credible interval for the mean is (0.67, 3.2)
- This interval reflects **posterior uncertainty**: how much the trend could plausibly be, given the observed data and model
- In principle this uncertainty is unrelated to the number of posterior draws

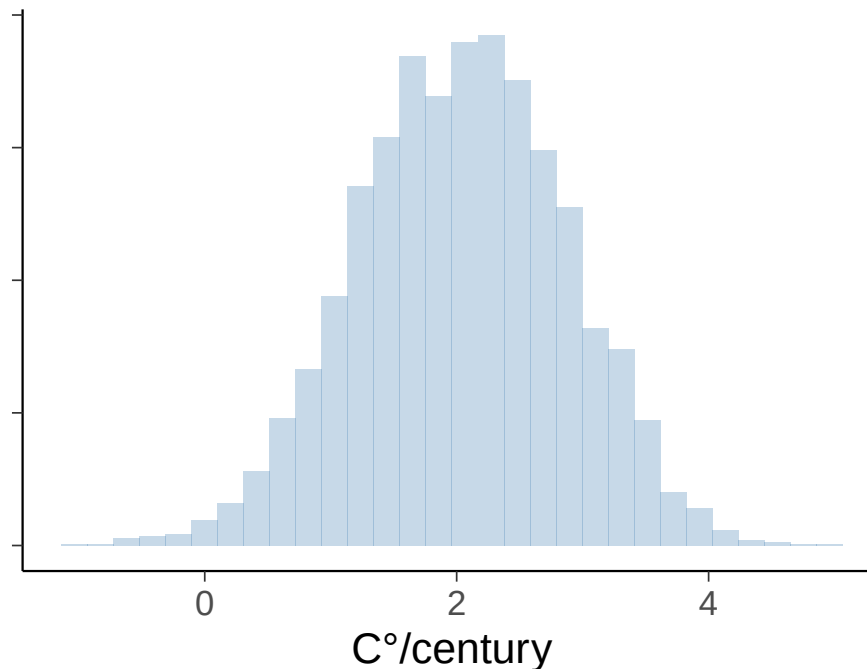
Posterior of temperature change



Example: Kilpisjärvi summer temperature

- But we got this result by sampling from the posterior distribution
- This introduces a second, separate source of uncertainty
 - how far our simulated mean $\hat{\theta}$ might be from the *true* posterior mean, simply because S is finite
- The Monte Carlo standard error (MCSE) quantifies exactly this

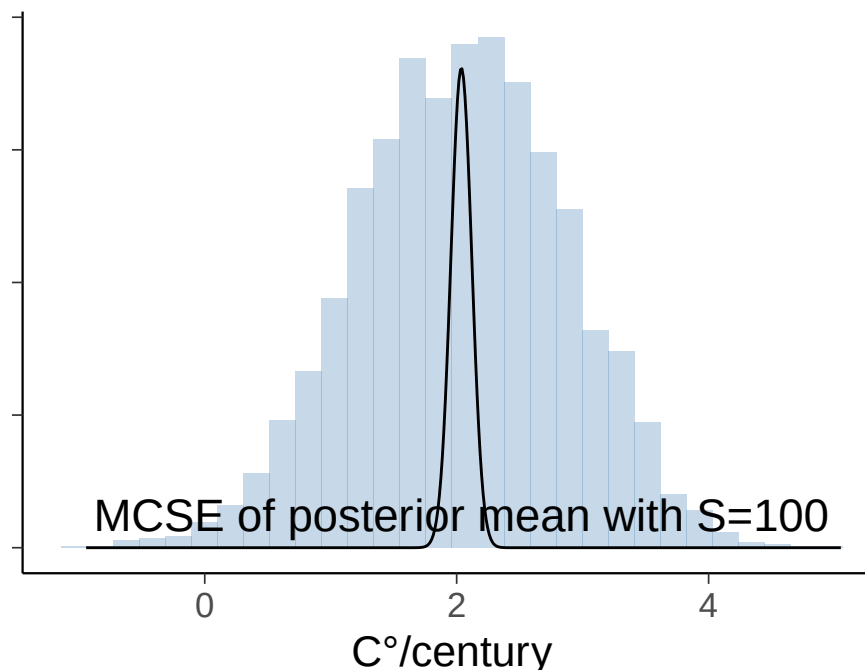
Posterior of temperature change



Example: Kilpisjärvi summer temperature

- Posterior SD, estimated from draws: $\sigma_{\theta} \approx 0.83$
- MCSE is $\frac{\sigma_{\theta}}{\sqrt{S}} \approx 0.083$ for $S = 100$
- By the CLT, the sampling distribution of $\hat{\theta}$ is approximately $N(\theta, \text{MCSE}^2)$
- And a 90% interval is then $1.928... \pm 1.645... \times 0.083... \approx (1.8, 2.1)$
- Using the same data, model and number of draws S . We'd expect our mean estimate to be within this range 90% of the time

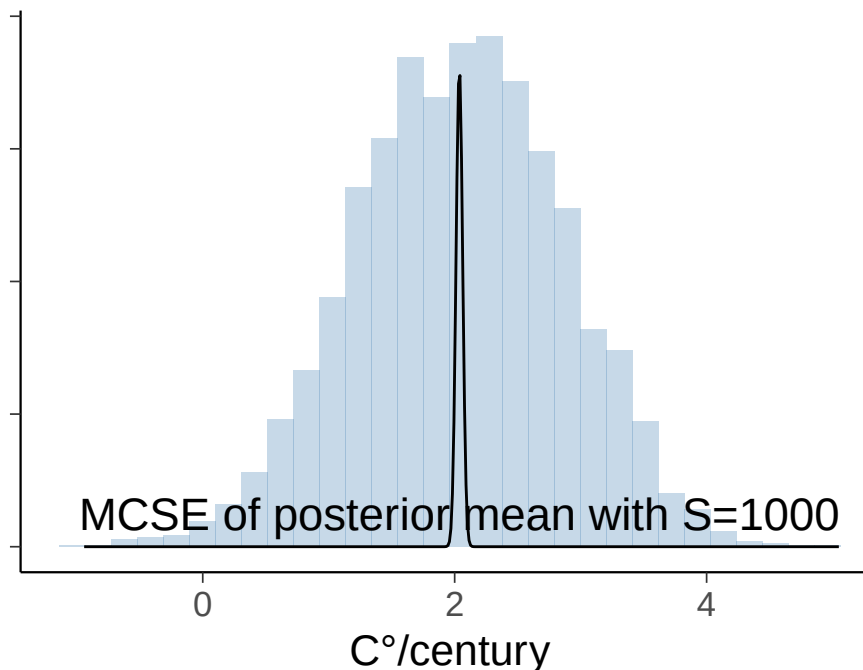
Posterior of temperature change



Example: Kilpisjärvi summer temperature

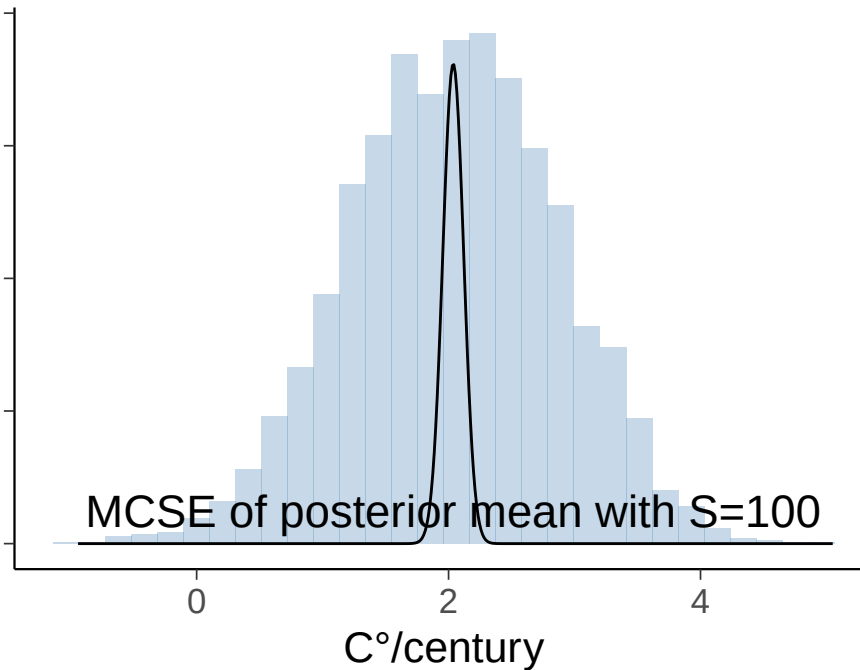
- Posterior SD, estimated from draws: $\sigma_{\theta} \approx 0.0263$
- MCSE is $\frac{\sigma_{\theta}}{\sqrt{S}} \approx 0.0026$ for $S = 1000$
- By the CLT, the sampling distribution of $\hat{\theta}$ is approximately $N(\theta, \text{MCSE}^2)$
- And a 90% interval is then $1.928... \pm 1.645... \times 0.026... \approx (1.9, 2.0)$
- Using the same data, model and number of draws S . We'd expect our mean estimate to be within this range 90% of the time

Posterior of temperature change

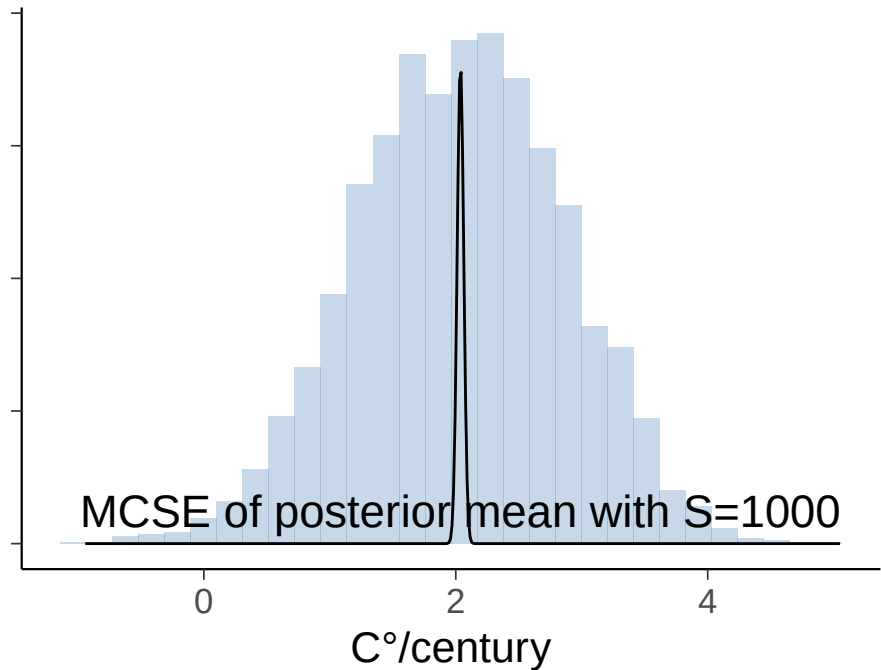


Example: Kilpisjärvi summer temperature

Posterior of temperature change

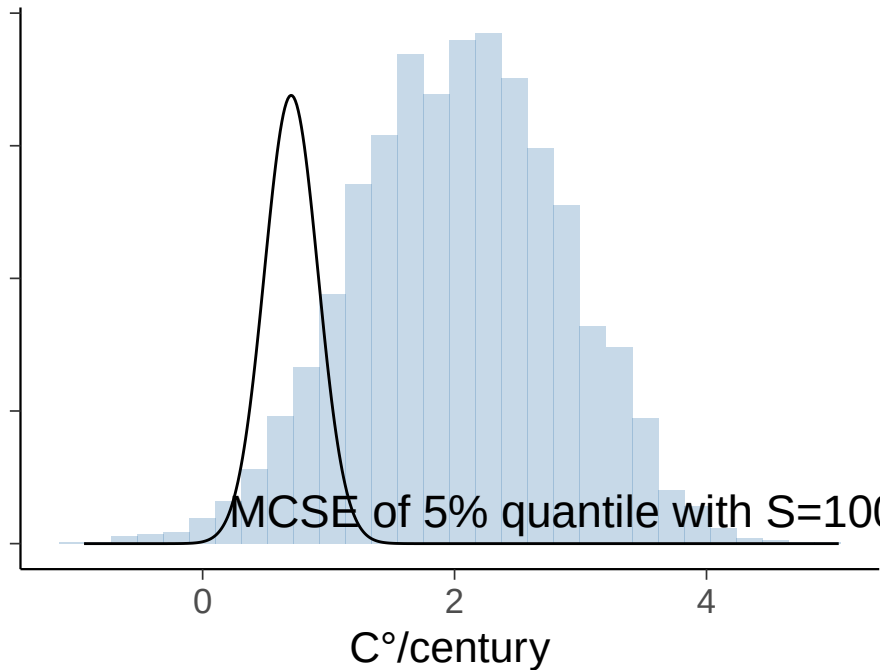


Posterior of temperature change

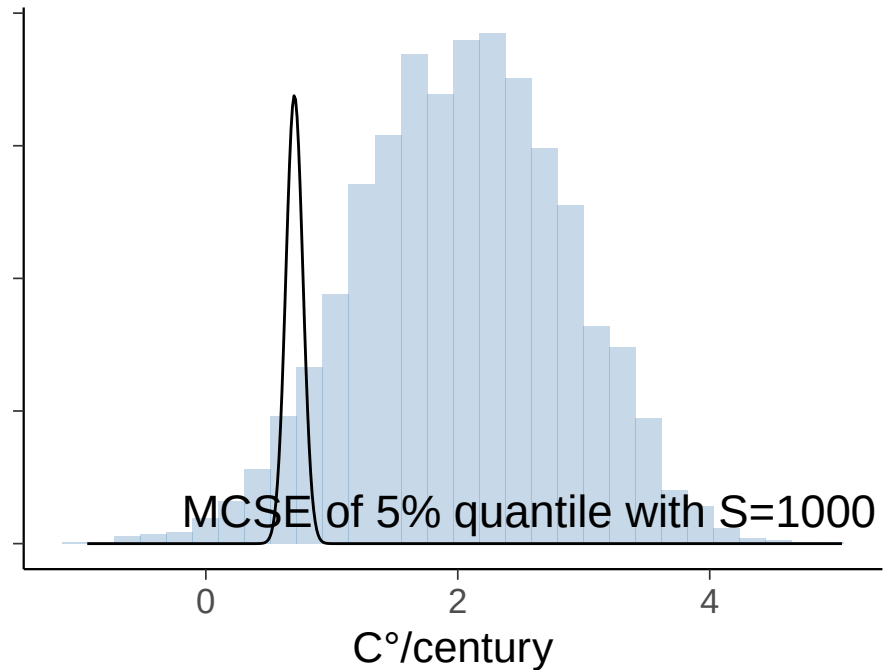


Example: Kilpisjärvi summer temperature

Posterior of temperature change



Posterior of temperature change



Tail quantiles are more difficult to estimate See [Vehtari, Gelman, Simpson, Carpenter, & Bürkner \(2021\)](#) for quantile MCSE computation. And Appendix at the end of these slides

How many digits to report?

- Too many digits make reading of the results more difficult and give false impression of accuracy

How many digits to report?

- Too many digits make reading of the results more difficult and give false impression of accuracy
- Use context, accuracy could change with the audience and intention

How many digits to report?

- Too many digits make reading of the results more difficult and give false impression of accuracy
- Use context, accuracy could change with the audience and intention
- Rule of thumb: 2 significant digits is usually enough; 1 is often fine early in an analysis

How many digits to report?

- Too many digits make reading of the results more difficult and give false impression of accuracy
- Use context, accuracy could change with the audience and intention
- Rule of thumb: 2 significant digits is usually enough; 1 is often fine early in an analysis
- Show meaningful digits given the posterior uncertainty

How many digits to report?

- Too many digits make reading of the results more difficult and give false impression of accuracy
- Use context, accuracy could change with the audience and intention
- Rule of thumb: 2 significant digits is usually enough; 1 is often fine early in an analysis
- Show meaningful digits given the posterior uncertainty
- Don't show digits which are just random noise (check MCSE)

How many digits to report?

- Too many digits make reading of the results more difficult and give false impression of accuracy
- Use context, accuracy could change with the audience and intention
- Rule of thumb: 2 significant digits is usually enough; 1 is often fine early in an analysis
- Show meaningful digits given the posterior uncertainty
- Don't show digits which are just random noise (check MCSE)
- Halving MCSE requires **4×** more draws → getting one more reliable digit is often more expensive than it looks

How many digits to report?

- Example: mean and 90% interval for temperature increase (°C/century)
 - **1.927743** and **[0.6656391, 3.1779750]** (NO!)
 - 1.9 and [0.7, 3.2] is ok
 - **2** and **[1, 3]** may also be ok

How many digits to report?

- Example: mean and 90% interval for temperature increase (°C/century)
 - **1.927743** and **[0.6656391, 3.1779750]** (NO!)
 - 1.9 and [0.7, 3.2] is ok
 - **2** and **[1, 3]** may also be ok
- Example: The probability that temp increase is positive
 - **0.9960000** (NO!)
 - **1.00** may also be ok
 - With 4000 draws $\text{MCSE} \approx 0.002$. We could report that probability is very likely larger than 0.99, or sample more to justify reporting three digits

Computing the Monte Carlo Standard Error

- `Posterior` package (in R) and `ArviZ` (in Python) provides functions to compute MCSE
- These functions are designed for MCMC draws (next week)
- If the number of draws is big (≥ 1000), then these are accurate enough for independent MC draws, too

Examples

Posterior mean and 5% and 95% quantiles:

```
draws |>
  subset_draws("beta100") |>
  summarize_draws(mean, ~quantile(.x, probs = c(0.05, 0.95)))
```

The corresponding MCSE estimates:

```
draws |>
  subset_draws("beta100") |>
  summarize_draws(mcse_mean,
    ~mcse_quantile(.x, probs = c(0.05, 0.95)))
```

Posterior probability $\text{beta100} > 0$ and the corresponding MCSE estimate:

```
draws |>
  mutate_variables(beta0p = beta100>0) |>
  subset_draws("beta0p") |>
  summarize_draws(mean, mcse = mcse_mean)
```

- In ArviZ (Python) use `az.summary` or `az.mcse` for similar functionality

Checking finite mean and variance

- We saw that we can not apply the CLT to Cauchy draws

Checking finite mean and variance

- We saw that we can not apply the CLT to Cauchy draws
- But we can always compute the mean and variance from a finite sample
 - MCSE computed from a finite sample can look small and reassuring, even when it's meaningless

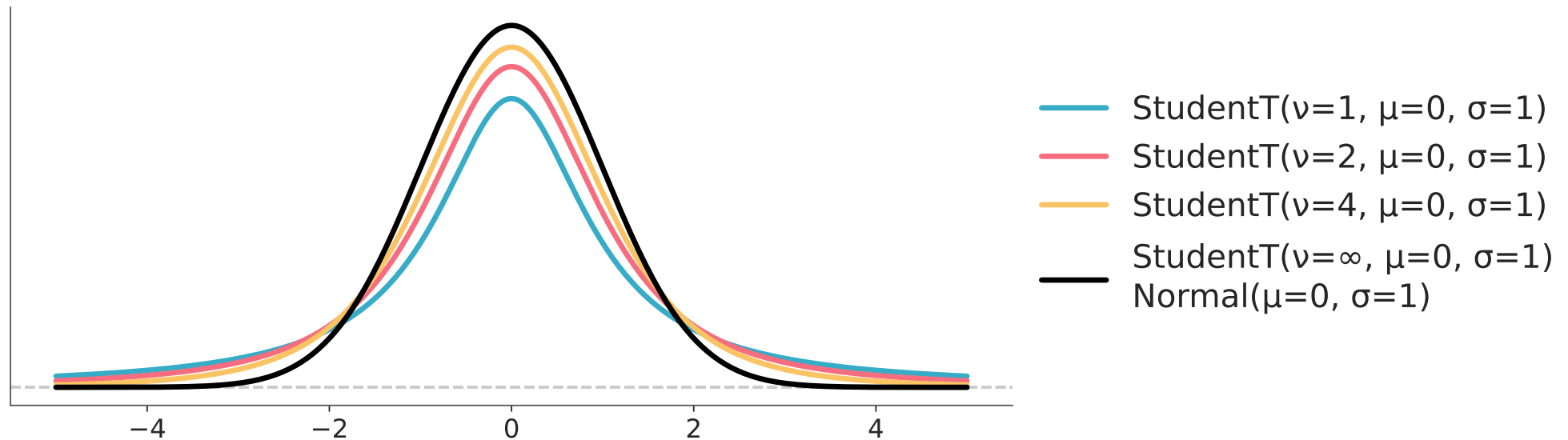
Checking finite mean and variance

- We saw that we can not apply the CLT to Cauchy draws
- But we can always compute the mean and variance from a finite sample
 - MCSE computed from a finite sample can look small and reassuring, even when it's meaningless
- The Pareto- $\hat{k}$ diagnostic tell us about the finite moments of a distribution

Checking finite mean and variance

- We saw that we can not apply the CLT to Cauchy draws
- But we can always compute the mean and variance from a finite sample
 - MCSE computed from a finite sample can look small and reassuring, even when it's meaningless
- The Pareto- $\hat{k}$ diagnostic tell us about the finite moments of a distribution
- If the mean and/or variance are non finite we can still report other summaries like the median, Median Absolute Deviation (MAD), interquartile range.

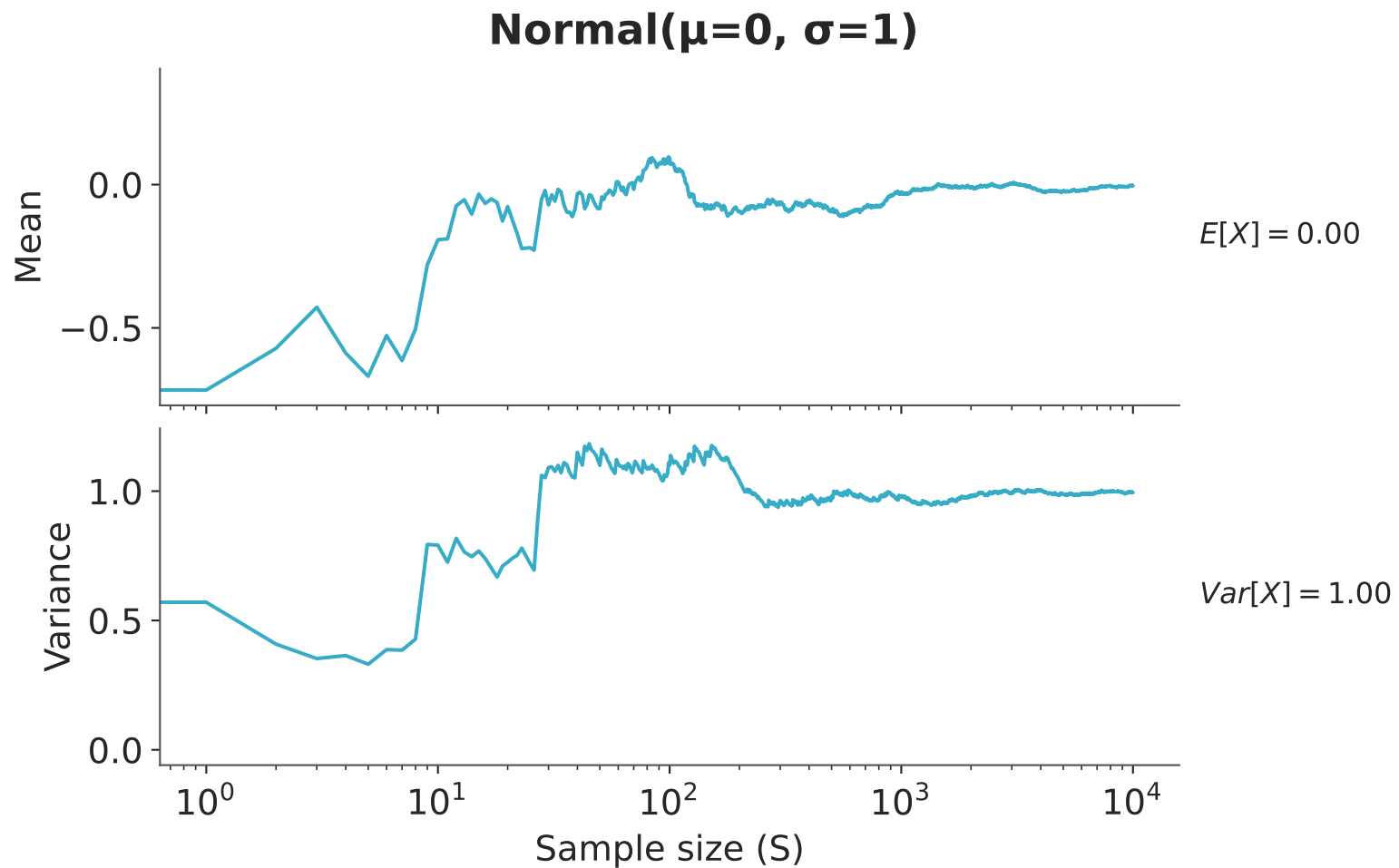
Simple example: $x \sim N, t_4, t_2, t_1$



- Normal: all moments finite
- StudentT: moments of order k are finite only for $\nu > k$

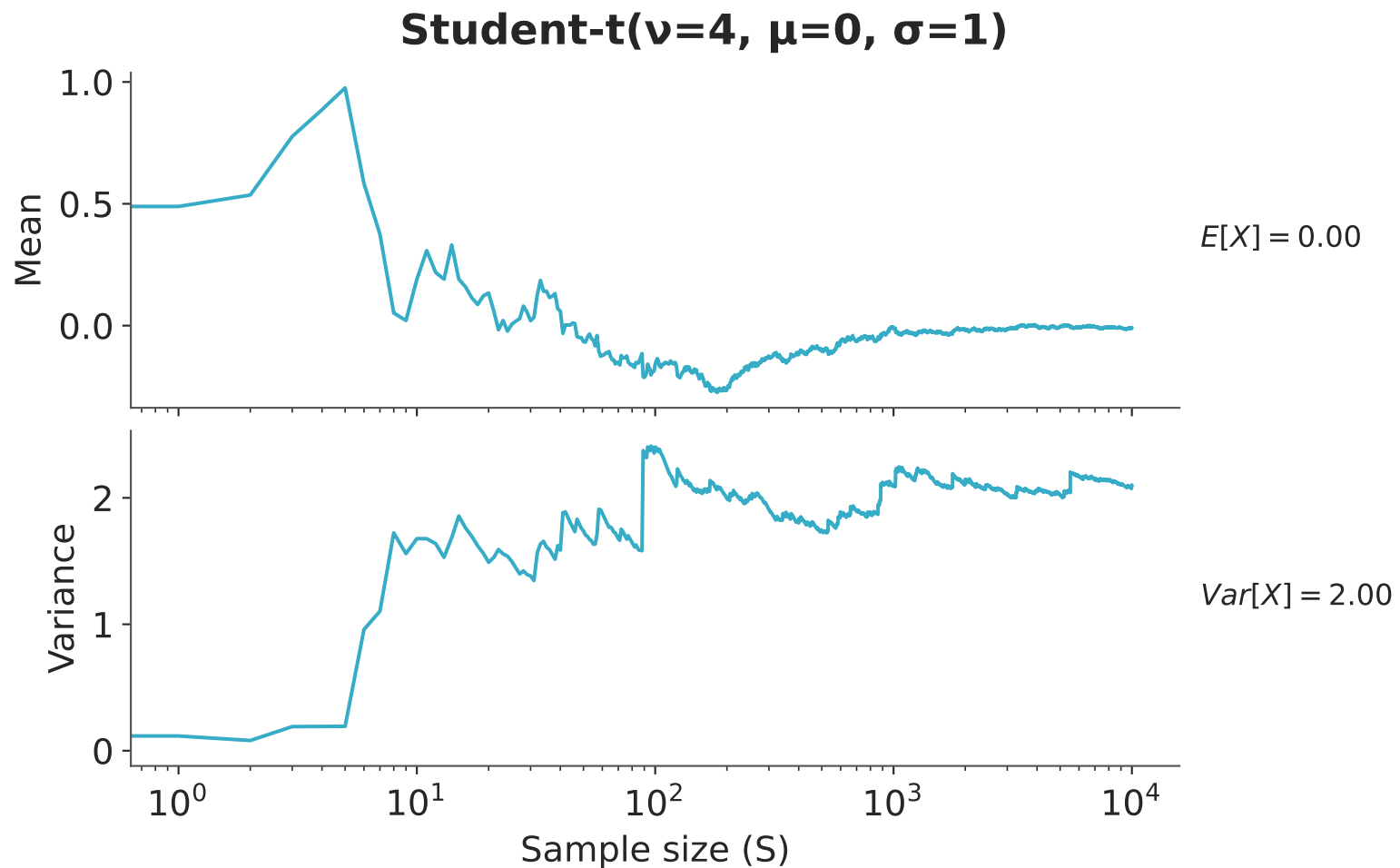
Simple example: $x \sim N$

- All moments finite



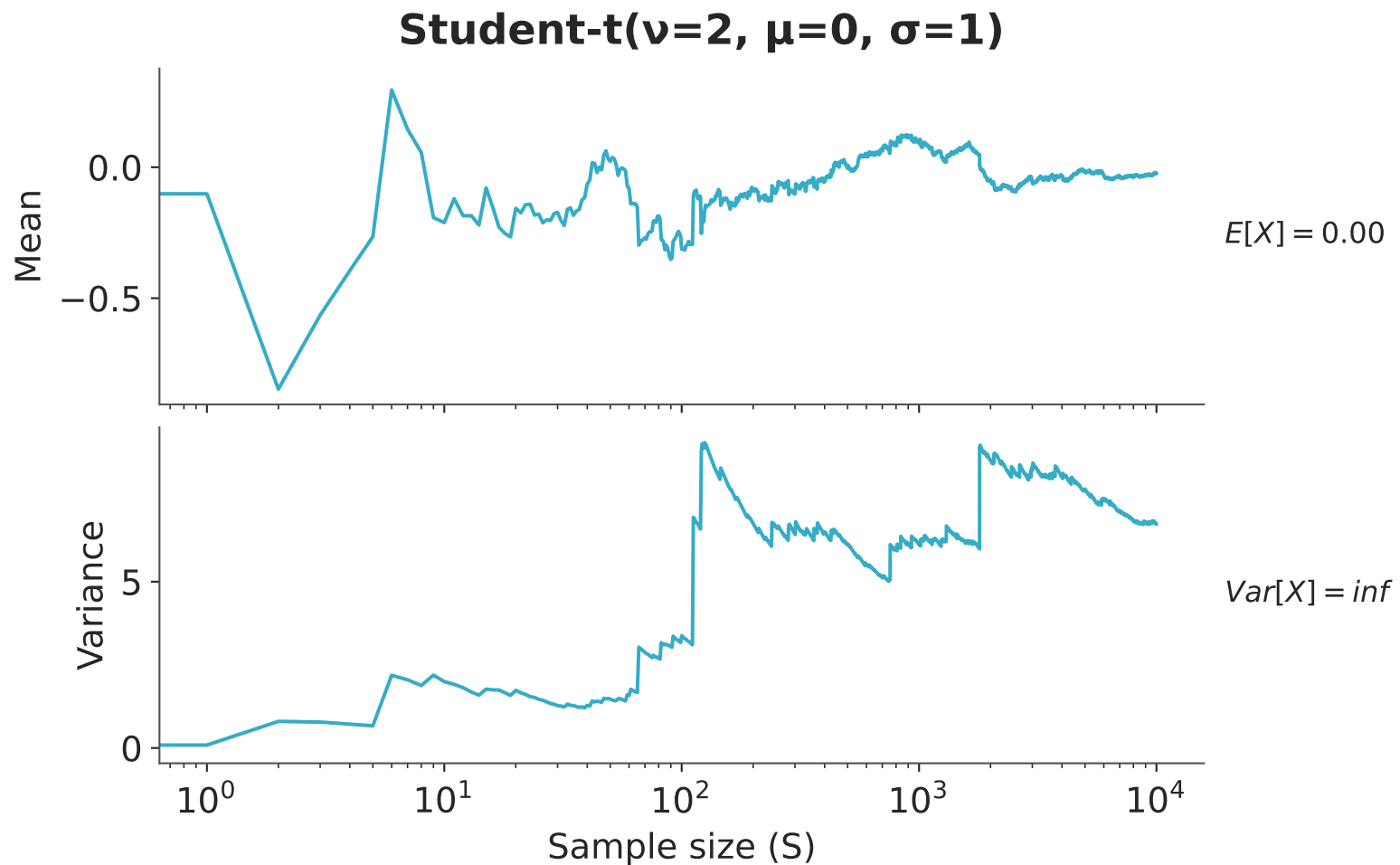
Simple example: $x \sim t_4$

- Only mean, variance, and skewness are finite



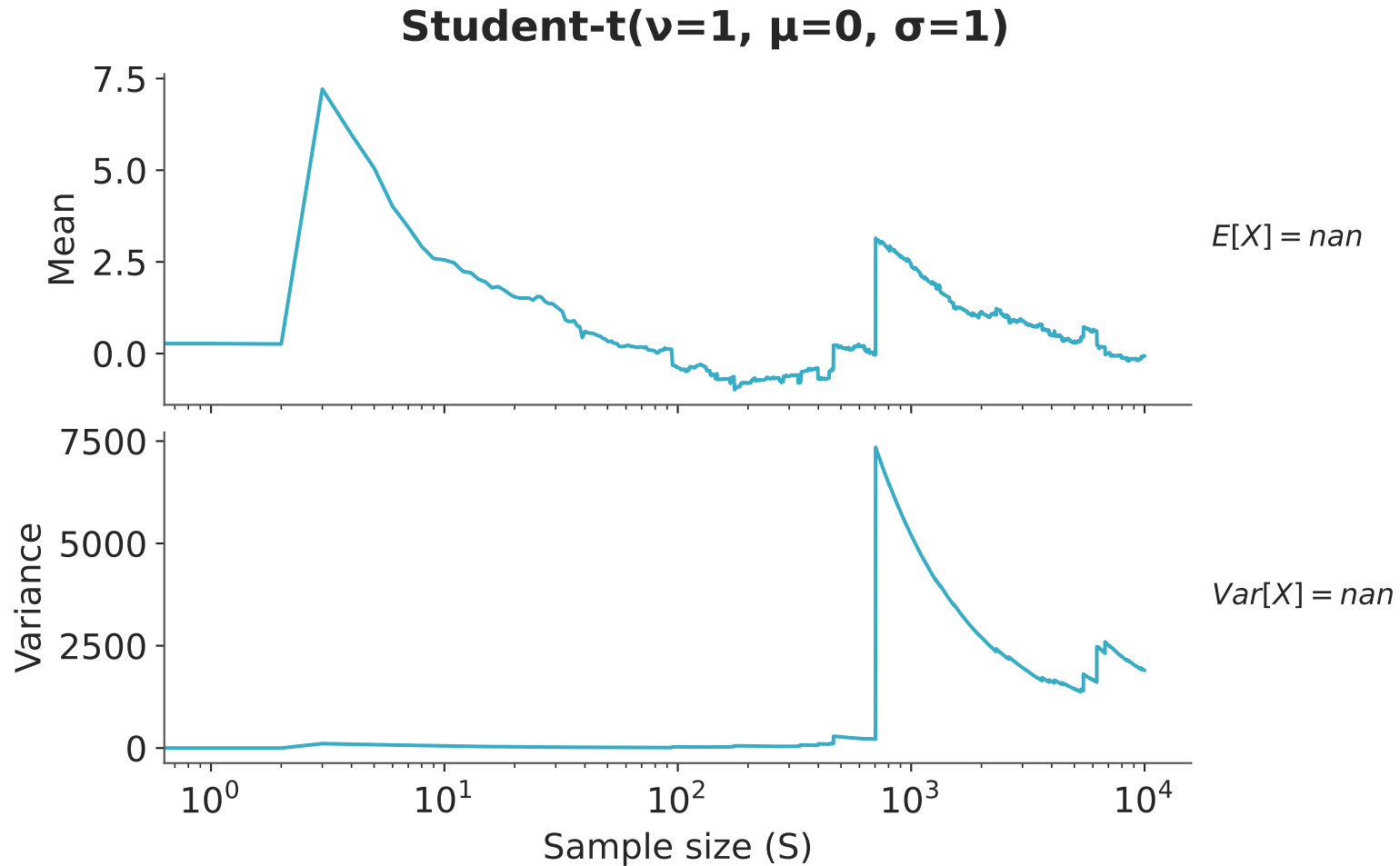
Simple example: $x \sim t_2$

- Only mean is finite



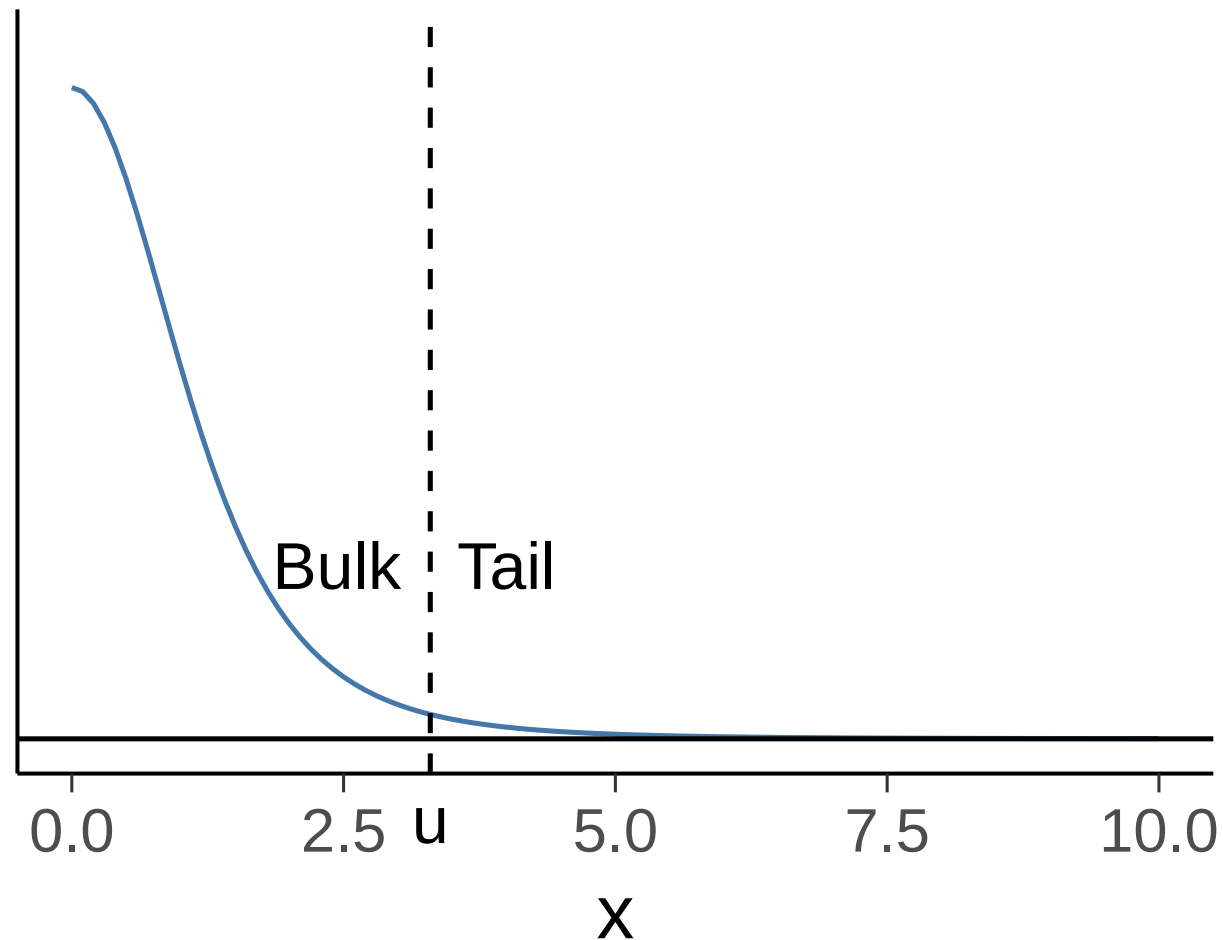
Simple example: $x \sim t_1$

- No moments are finite



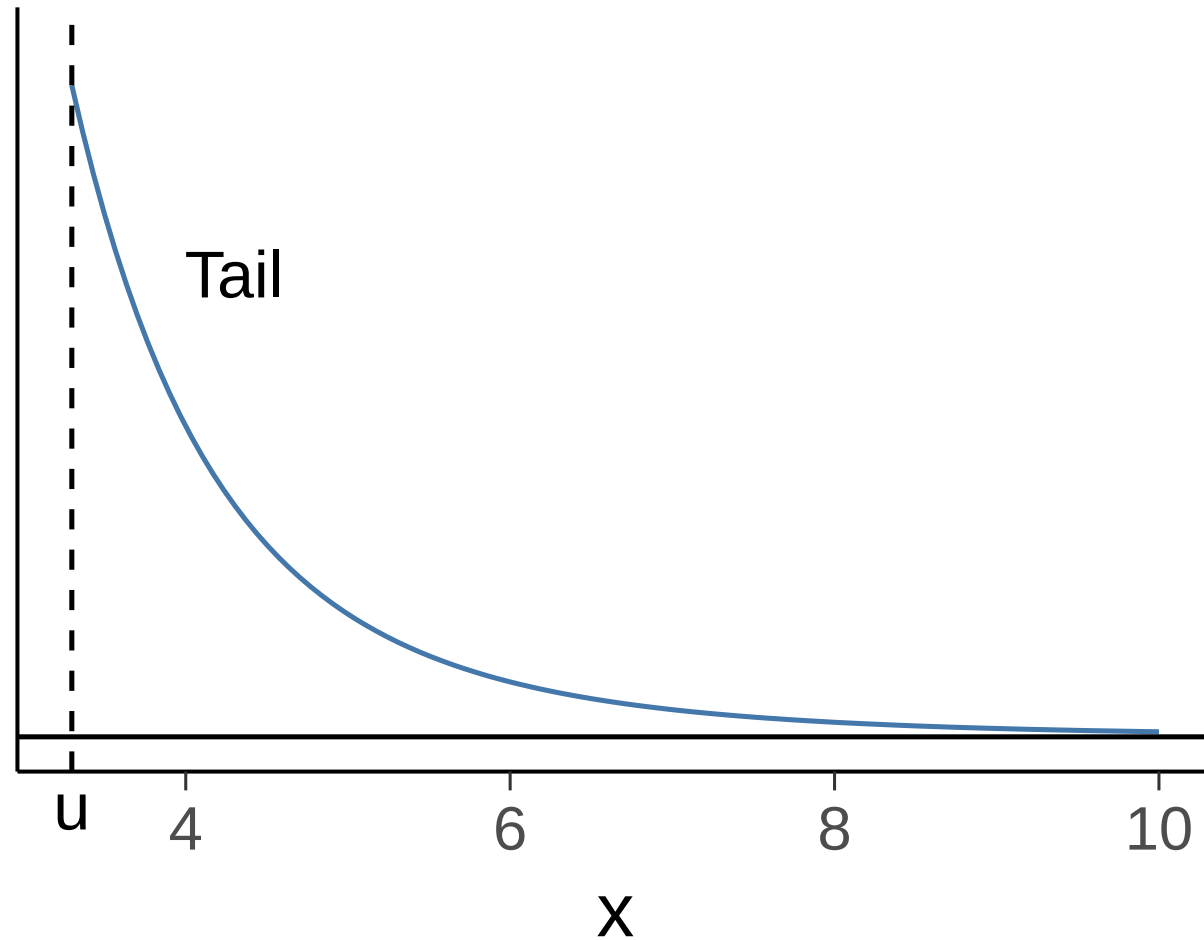
Pareto- $\hat{k}$ diagnostic

For many distributions the tail ($x > u$) is well approximated with Generalized Pareto distribution (GPD)



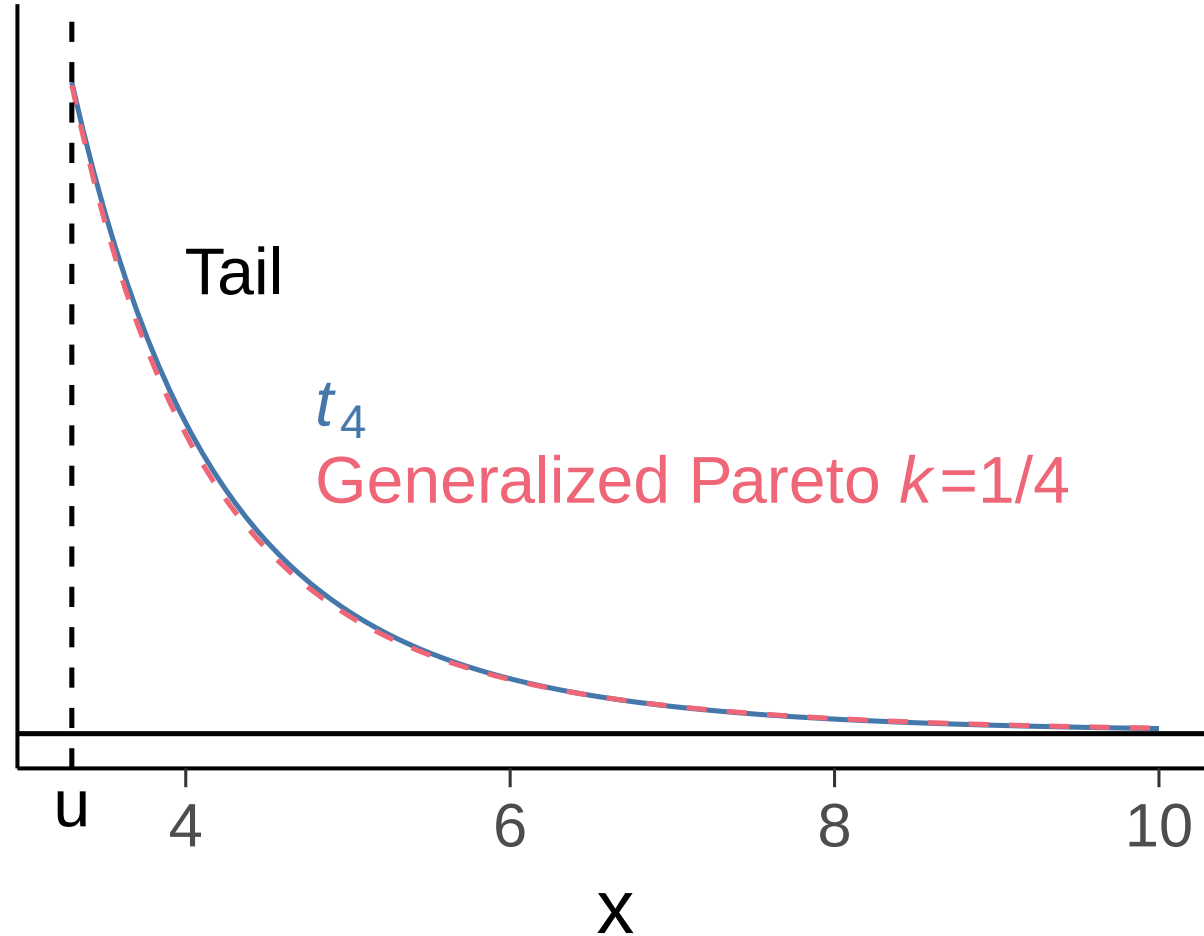
Pareto- $\hat{k}$ diagnostic

For many distributions the tail ($x > u$) is well approximated with Generalized Pareto distribution (GPD)



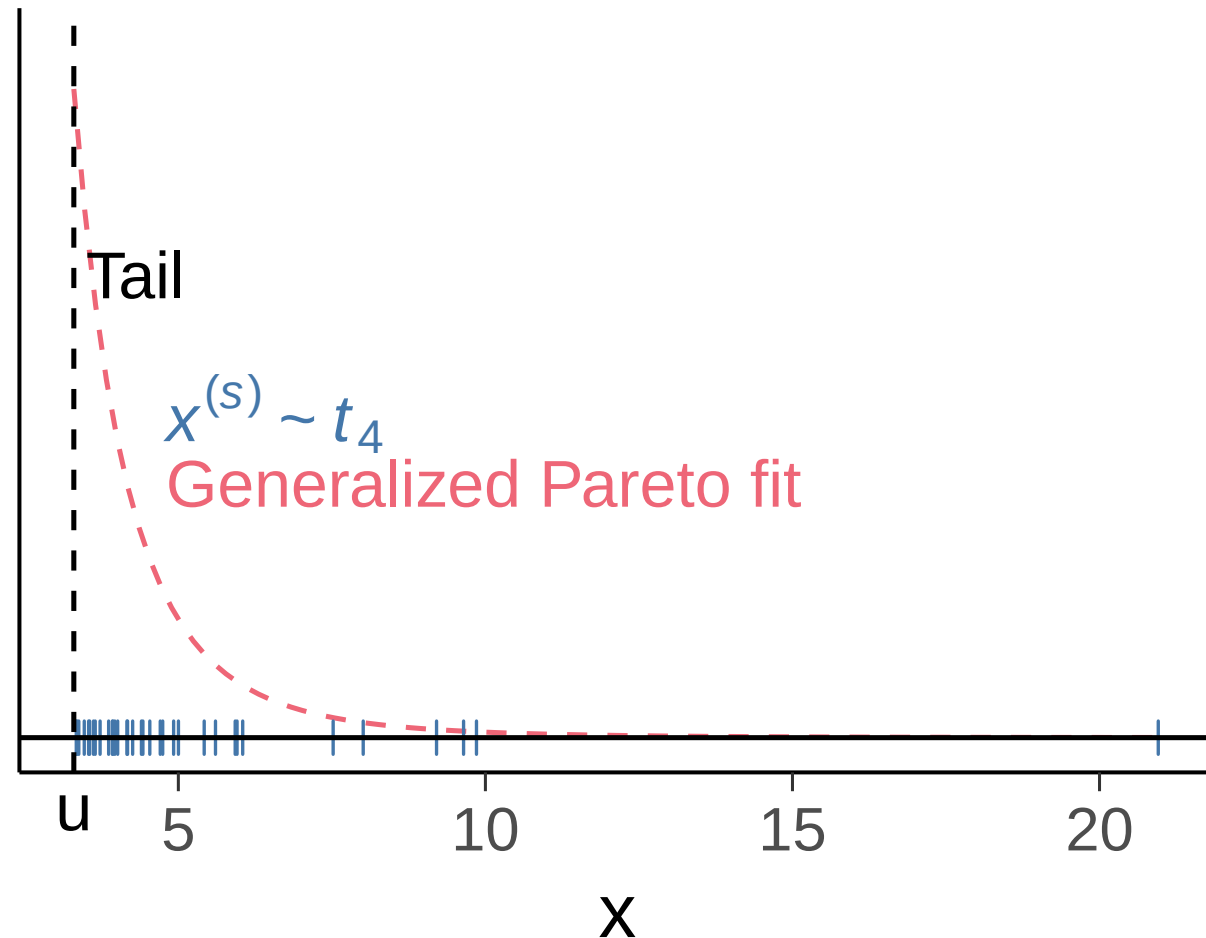
Pareto- $\hat{k}$ diagnostic

GPD has a shape parameter k , and $\frac{1}{k}$ finite fractional moments

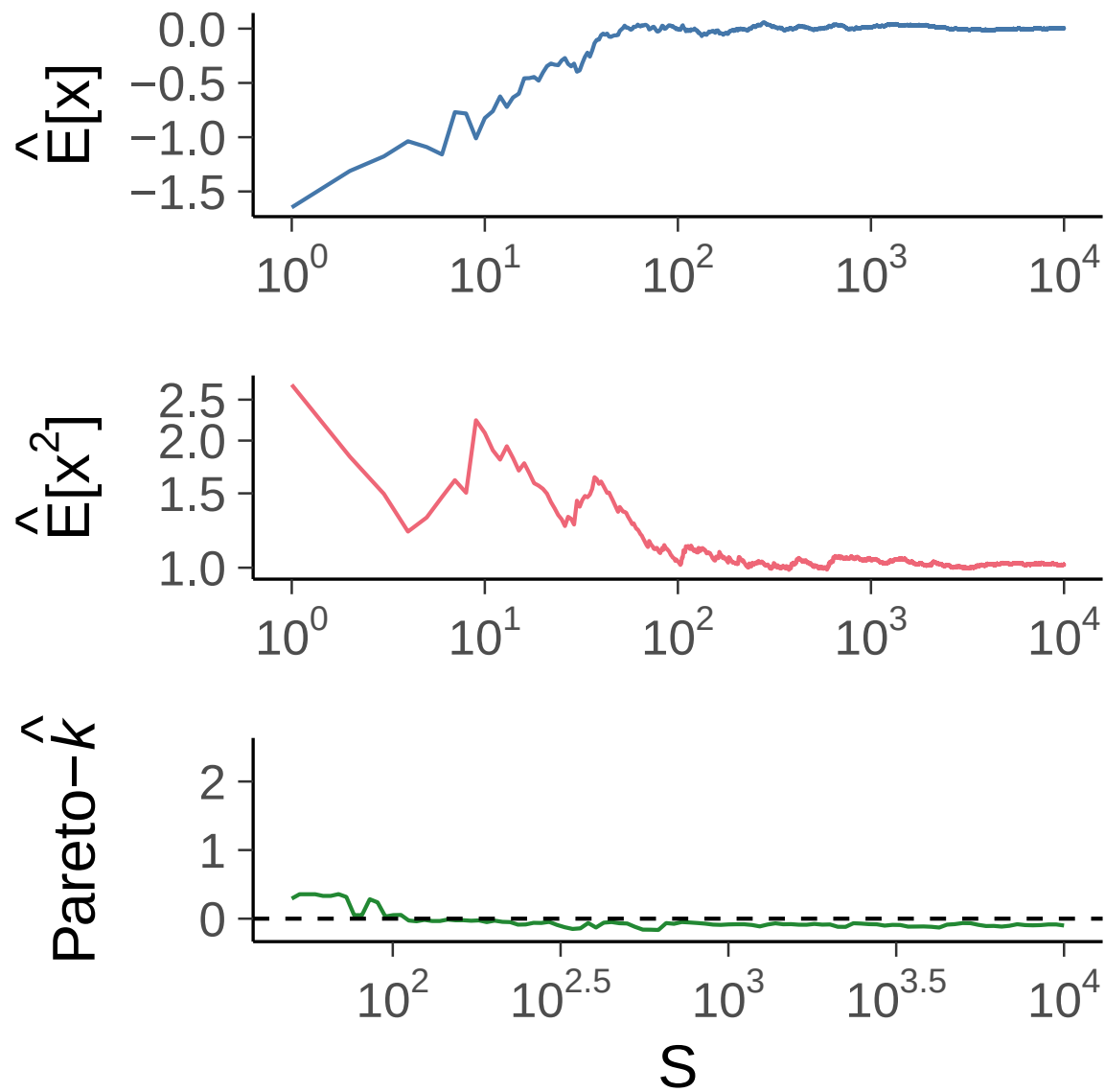


Pareto- $\hat{k}$ diagnostic

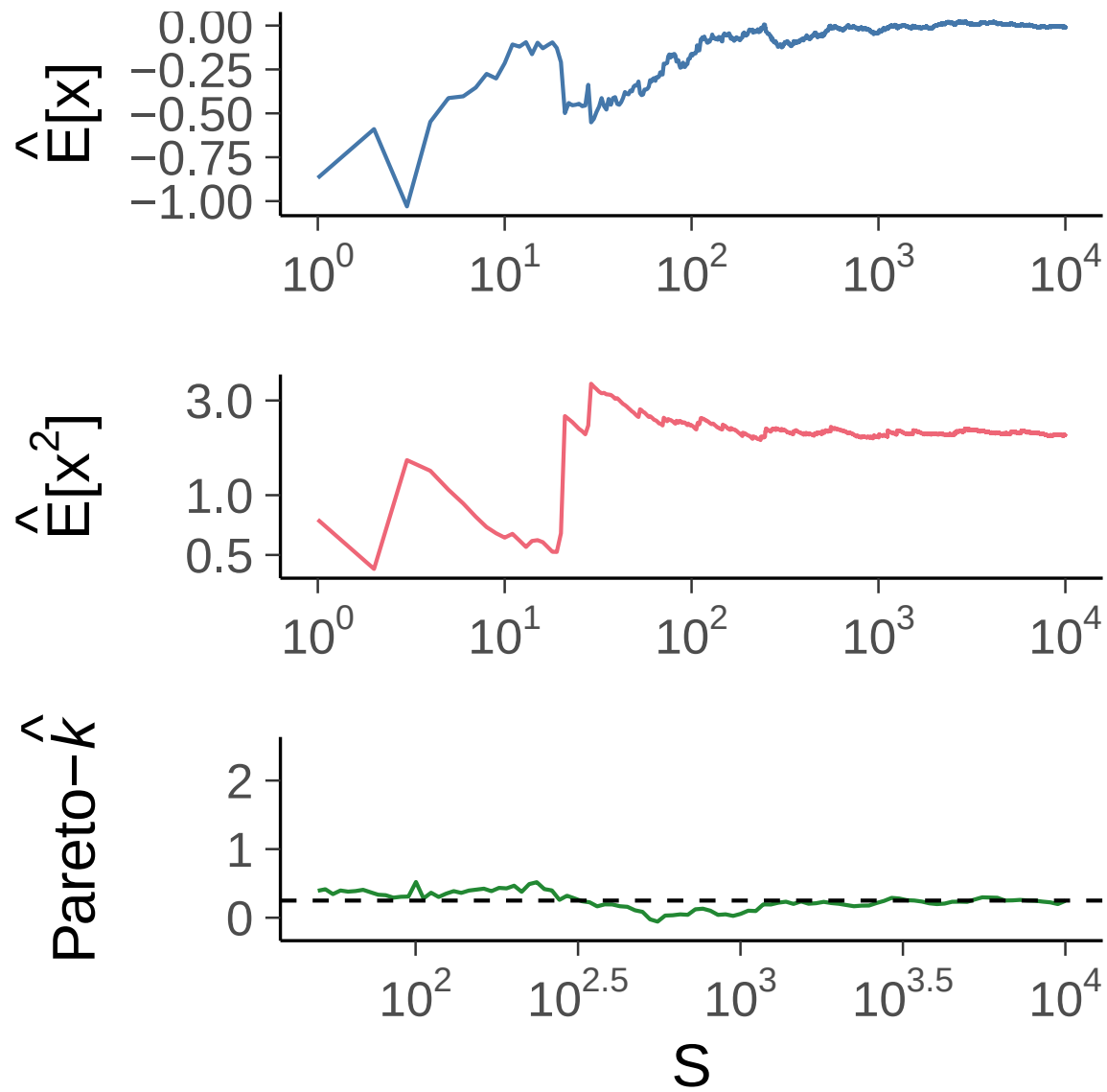
In practice we estimate the shape parameter k from the observed tail data



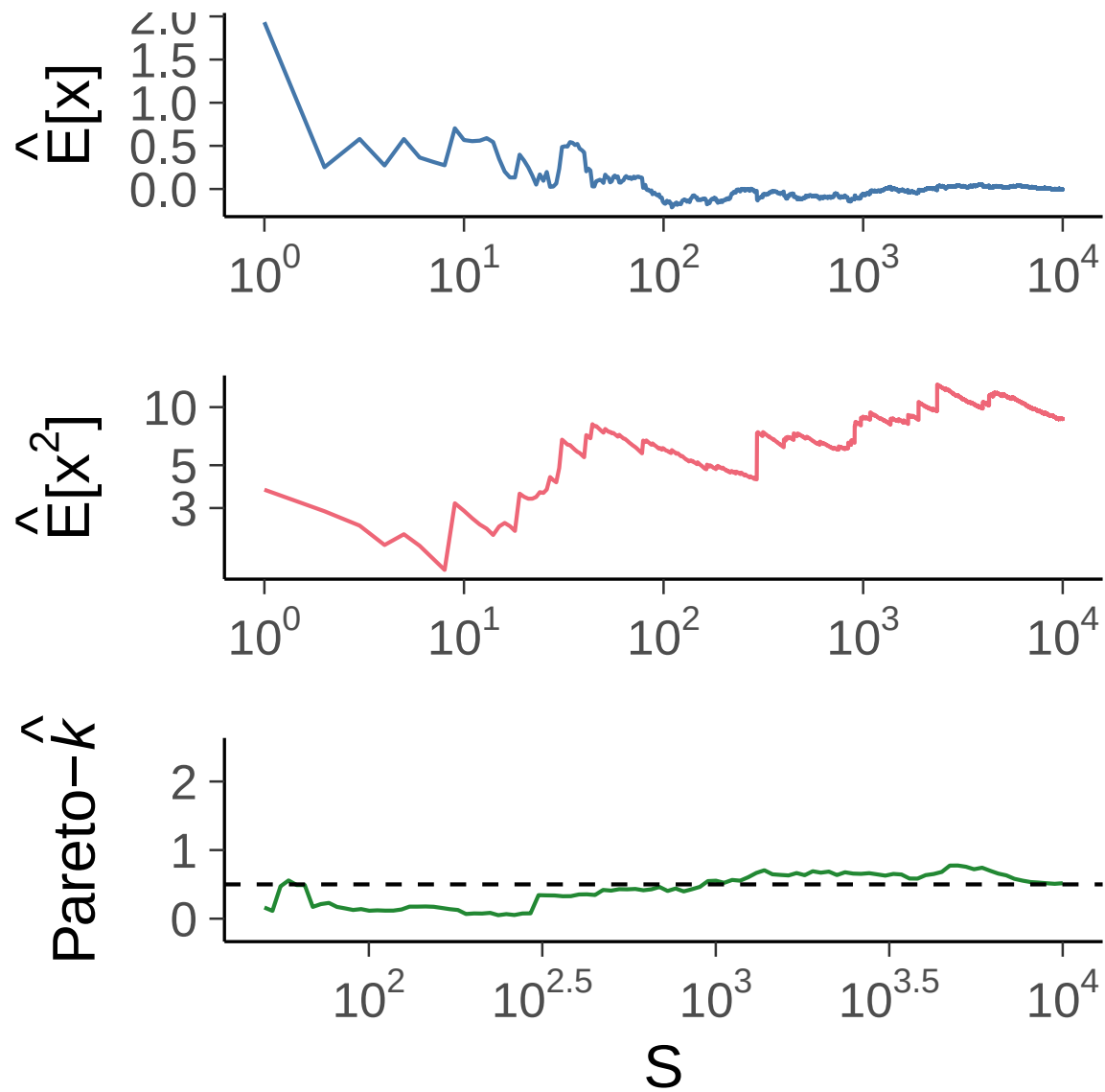
Pareto- $\hat{k}$ diagnostic: $x \sim N$



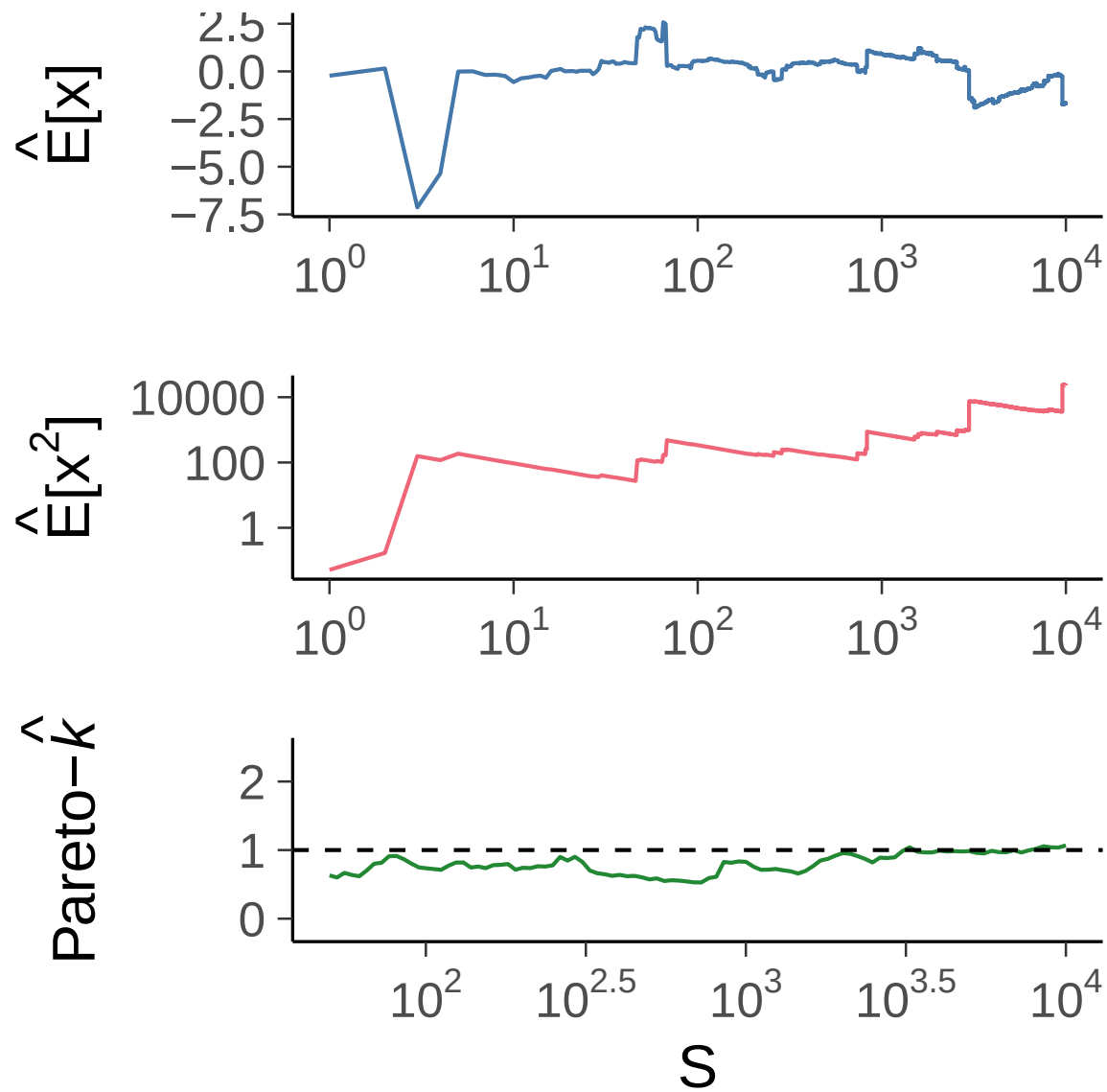
Pareto- $\hat{k}$ diagnostic: $x \sim t_4$



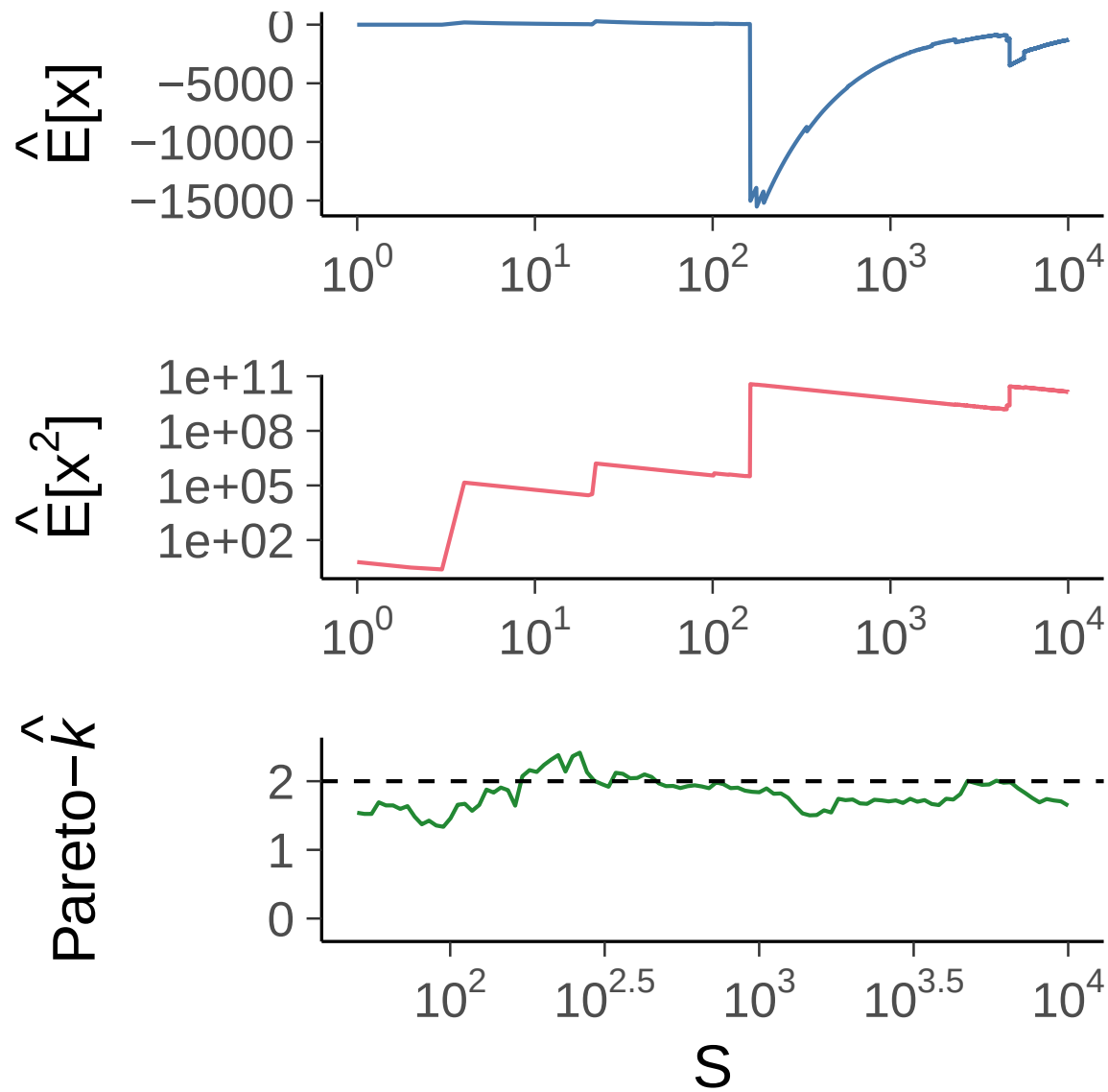
Pareto- $\hat{k}$ diagnostic: $x \sim t_2$



Pareto- $\hat{k}$ diagnostic: $x \sim t_1$



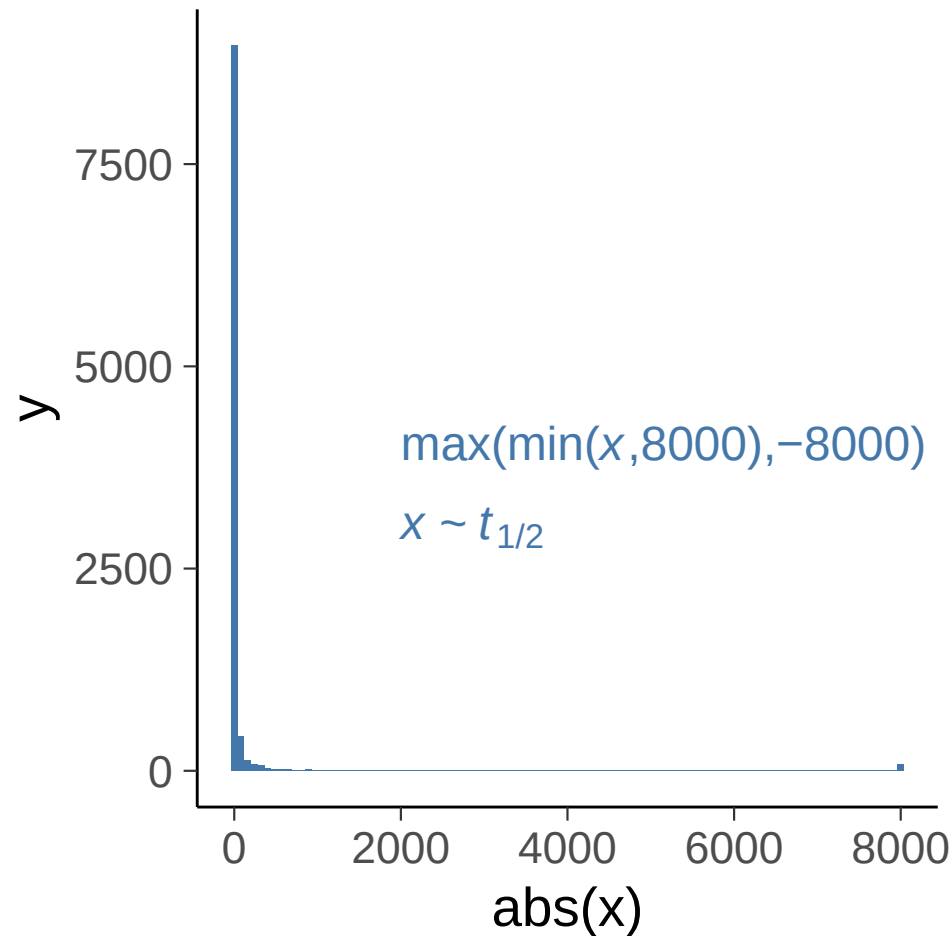
Pareto- $\hat{k}$ diagnostic: $x \sim t_{-}(\frac{1}{2})$



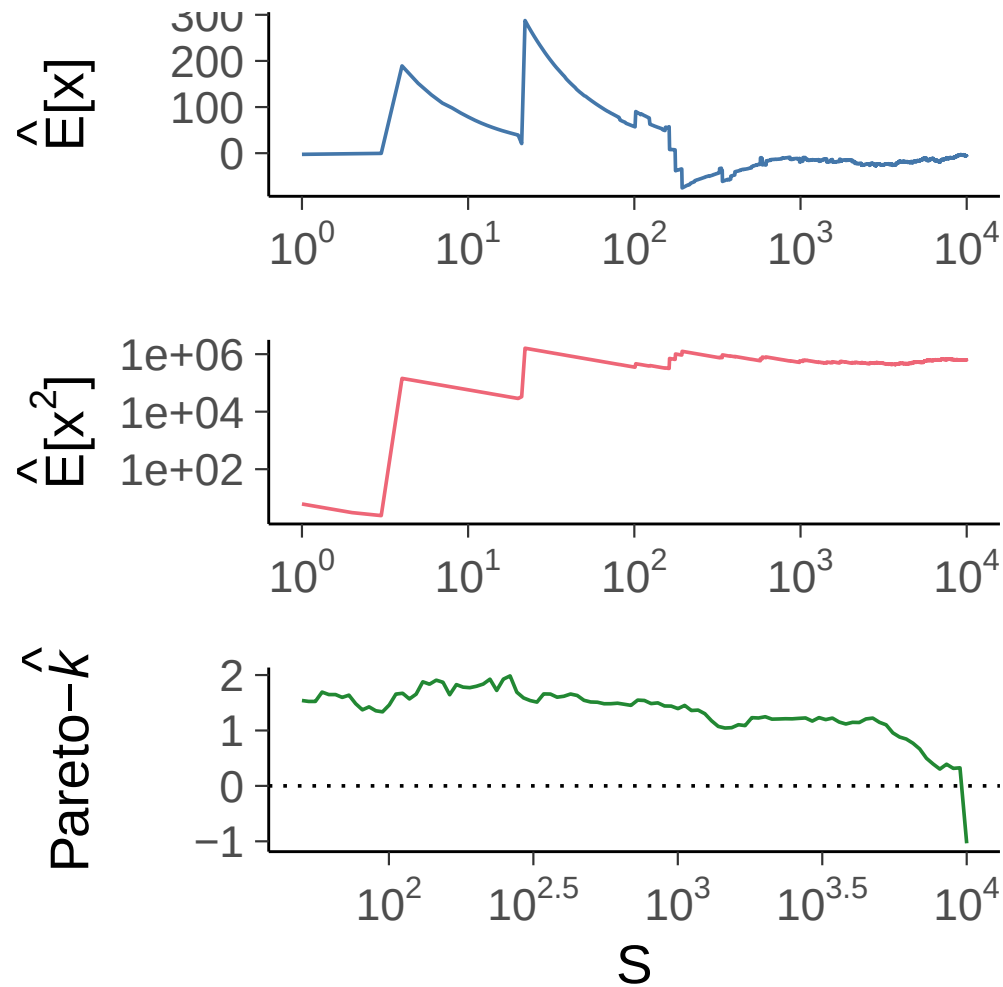
Pareto- $\hat{k}$ diagnostic is pre-asymptotic diagnostic

Thick tailed but truncated distribution

We can make estimates only based on what we have observed.



Pareto- $\hat{k}$ diagnostic: thick-tailed bounded distribution



Pareto- $\hat{k}$ in posterior package

```
> drt |> summarise_draws(mean, sd, mcse_mean)
```

variable	mean	sd	mcse_mean
xn	0.007	0.99	0.01
xt3	0.004	1.66	0.02
xt2_5	0.002	2.01	0.02
xt2	-0.008	3.00	0.03
xt1_5	-0.067	8.14	0.08
xt1	-1.57	122.	1.21

Pareto- $\hat{k}$ in posterior package

```
> drt |> summarise_draws(mean, sd, mcse_mean, pareto_khat)
```

variable	mean	sd	mcse_mean	pareto_khat
xn	0.007	0.99	0.01	-0.02
xt3	0.004	1.66	0.02	0.36
xt2_5	0.002	2.01	0.02	0.43
xt2	-0.008	3.00	0.03	0.53
xt1_5	-0.067	8.14	0.08	0.72
xt1	-1.57	122.	1.21	1.08

Pareto- $\hat{k}$ diagnostic use

- To check posterior of any quantity of interest
 - if high $\hat{k}$, maybe use some other summary than mean, e.g., quantiles

See more in Vehtari, Simpson, Gelman, Yao, and Gabry (2024). Pareto smoothed importance sampling. *JMLR*, 25(72):1-58.

Pareto- $\hat{k}$ diagnostic use

- To check posterior of any quantity of interest
 - if high $\hat{k}$, maybe use some other summary than mean, e.g., quantiles
- Especially useful inside algorithms that rely on expectations
 - automated diagnostic as in PSIS-LOO (Lecture 9) and `priorsense` (lecture 11)

See more in Vehtari, Simpson, Gelman, Yao, and Gabry (2024). Pareto smoothed importance sampling. *JMLR*, 25(72):1-58.

Pareto- $\hat{k}$ diagnostic use

- To check posterior of any quantity of interest
 - if high $\hat{k}$, maybe use some other summary than mean, e.g., quantiles
- Especially useful inside algorithms that rely on expectations
 - automated diagnostic as in PSIS-LOO (Lecture 9) and `priorsense` (lecture 11)
- $\hat{k}$ estimate has it's own variation given finite sample size
 - e.g. if close to 0.5 more draws help to improve to decide whether $k < 0.5$

See more in Vehtari, Simpson, Gelman, Yao, and Gabry (2024). Pareto smoothed importance sampling. *JMLR*, 25(72):1-58.

Pareto- $\hat{k}$ diagnostic use

- To check posterior of any quantity of interest
 - if high $\hat{k}$, maybe use some other summary than mean, e.g., quantiles
- Especially useful inside algorithms that rely on expectations
 - automated diagnostic as in PSIS-LOO (Lecture 9) and `priorsense` (lecture 11)
- $\hat{k}$ estimate has it's own variation given finite sample size
 - e.g. if close to 0.5 more draws help to improve to decide whether $k < 0.5$
- On Lecture 9 we will see a new method called Pareto-smoothing, which improves the mean estimate when Pareto- $k < 0.7$.

See more in Vehtari, Simpson, Gelman, Yao, and Gabry (2024). Pareto smoothed importance sampling. *JMLR*, 25(72):1-58.

Direct simulation

- So far we've assumed we can simulate directly from the target distribution
 - Standard distributions (normal, uniform, exponential, ...) have well-known direct simulation methods
 - Some derived quantities (e.g. sums, transformations of these) also do

Direct simulation

- So far we've assumed we can simulate directly from the target distribution
 - Standard distributions (normal, uniform, exponential, ...) have well-known direct simulation methods
 - Some derived quantities (e.g. sums, transformations of these) also do
- This assumes we can generate independent draws from the target
 - In practice we rely on pseudo-random number generators
 - Modern statistical software has good, well-tested pseudo-RNGs, this is not the bottleneck

Direct simulation

- So far we've assumed we can simulate directly from the target distribution
 - Standard distributions (normal, uniform, exponential, ...) have well-known direct simulation methods
 - Some derived quantities (e.g. sums, transformations of these) also do
- This assumes we can generate independent draws from the target
 - In practice we rely on pseudo-random number generators
 - Modern statistical software has good, well-tested pseudo-RNGs, this is not the bottleneck
- The problem: we often don't know how to directly simulate from the target distribution
 - Especially true for posterior distributions, which usually have no standard form
 - We need indirect sampling methods (or some approximation)

Indirect sampling

- Rejection sampling
- Importance sampling
- Markov chain Monte Carlo (next week)

Rejection sampling: back to the π example

- Recall: sample uniformly from the square, keep only points inside the circle

Rejection sampling: back to the π example

- Recall: sample uniformly from the square, keep only points inside the circle
- Two distributions are involved:
 - An easy one we can sample from directly $\rightarrow$ uniform on the square
 - A harder one we actually want draws from $\rightarrow$ uniform on the disk

Rejection sampling: back to the π example

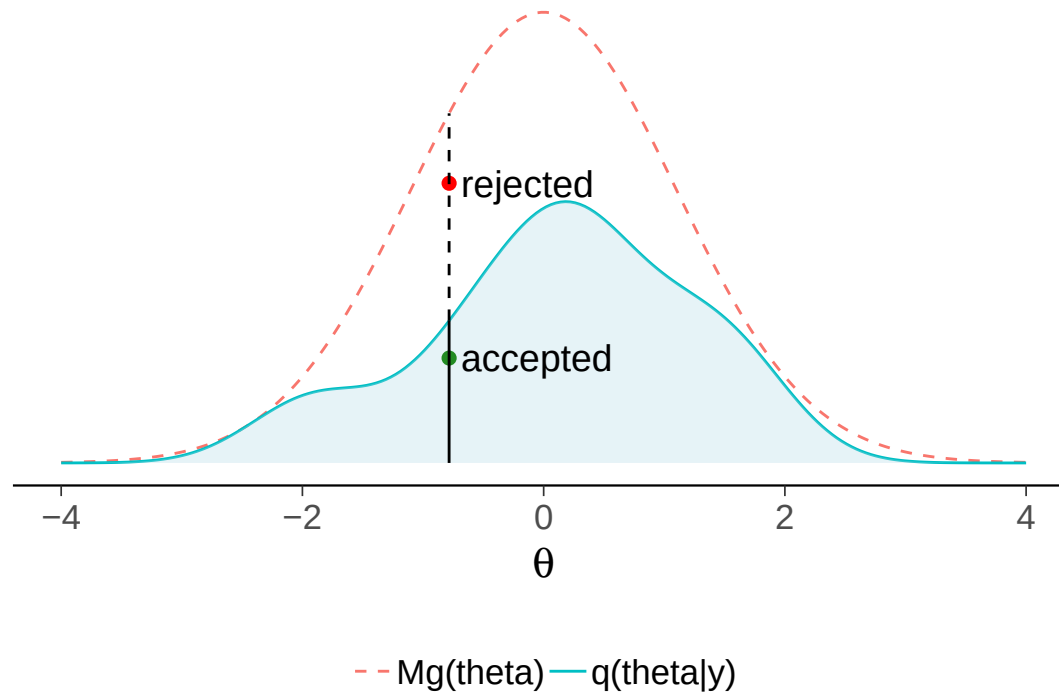
- Recall: sample uniformly from the square, keep only points inside the circle
- Two distributions are involved:
 - An easy one we can sample from directly $\rightarrow$ uniform on the square
 - A harder one we actually want draws from $\rightarrow$ uniform on the disk
- We sampled from the easy one, then accepted a draw only if it landed in the region matching the hard one

Rejection sampling: back to the π example

- Recall: sample uniformly from the square, keep only points inside the circle
- Two distributions are involved:
 - An easy one we can sample from directly $\rightarrow$ uniform on the square
 - A harder one we actually want draws from $\rightarrow$ uniform on the disk
- We sampled from the easy one, then accepted a draw only if it landed in the region matching the hard one
- The accepted points are exact draws from the target

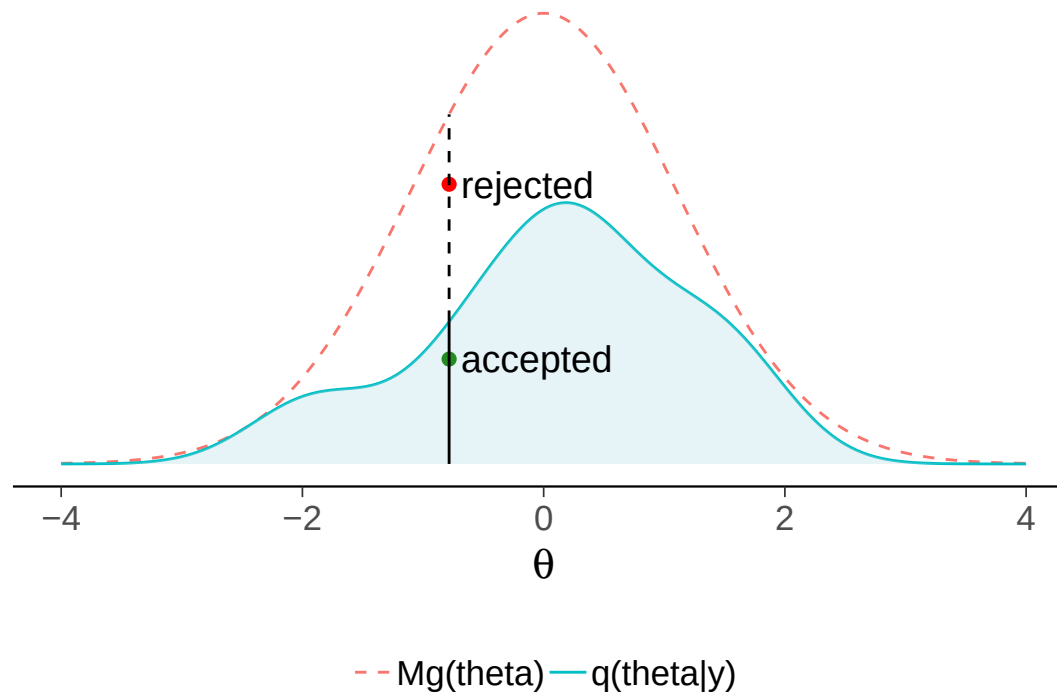
Rejection sampling

- Proposal forms envelope over the target distribution $\frac{q(\theta|y)}{Mg(\theta)} \leq 1$



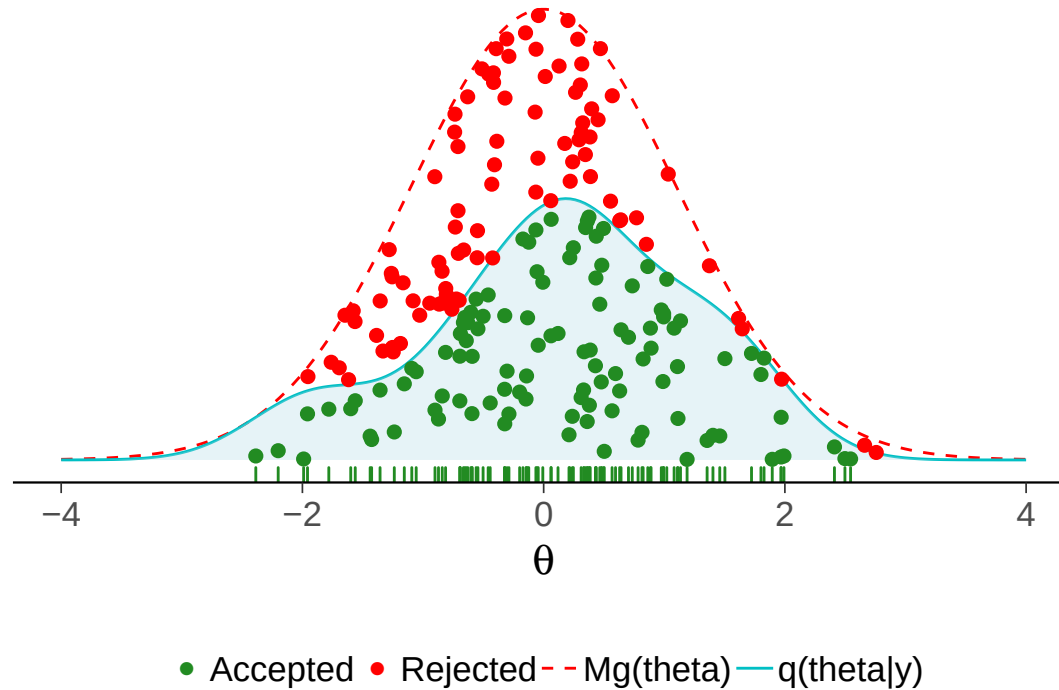
Rejection sampling

- Proposal forms envelope over the target distribution $\frac{q(\theta|y)}{Mg(\theta)} \leq 1$
- Draw from the proposal and accept with probability $\frac{q(\theta|y)}{Mg(\theta)}$



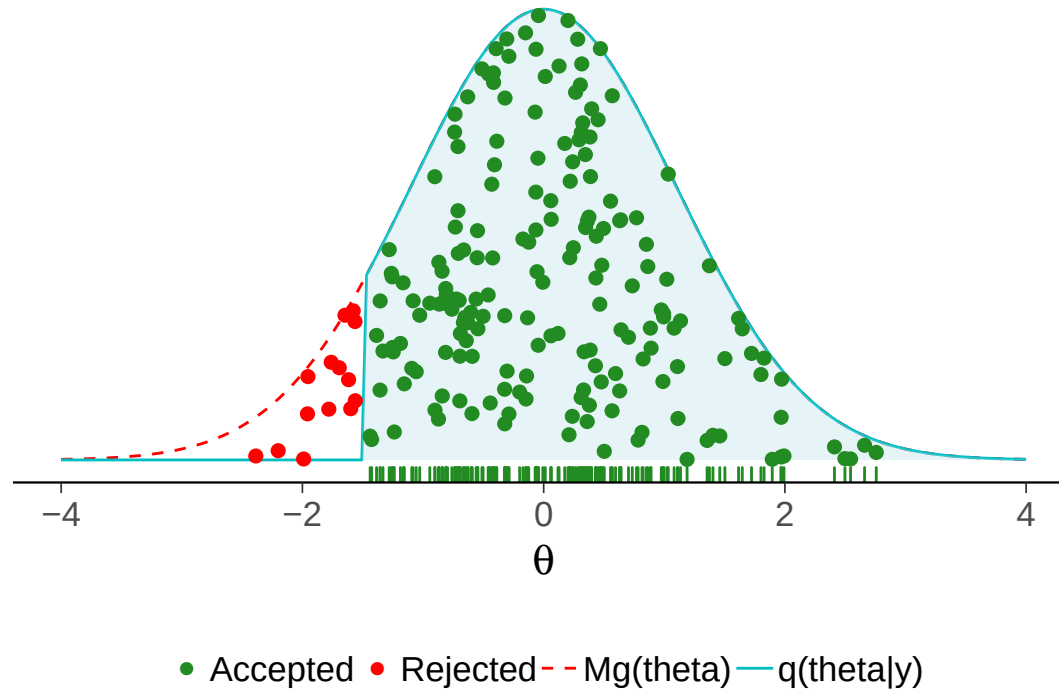
Rejection sampling

- Repeat for large number of trials



Rejection sampling

- Very effective for truncated distributions

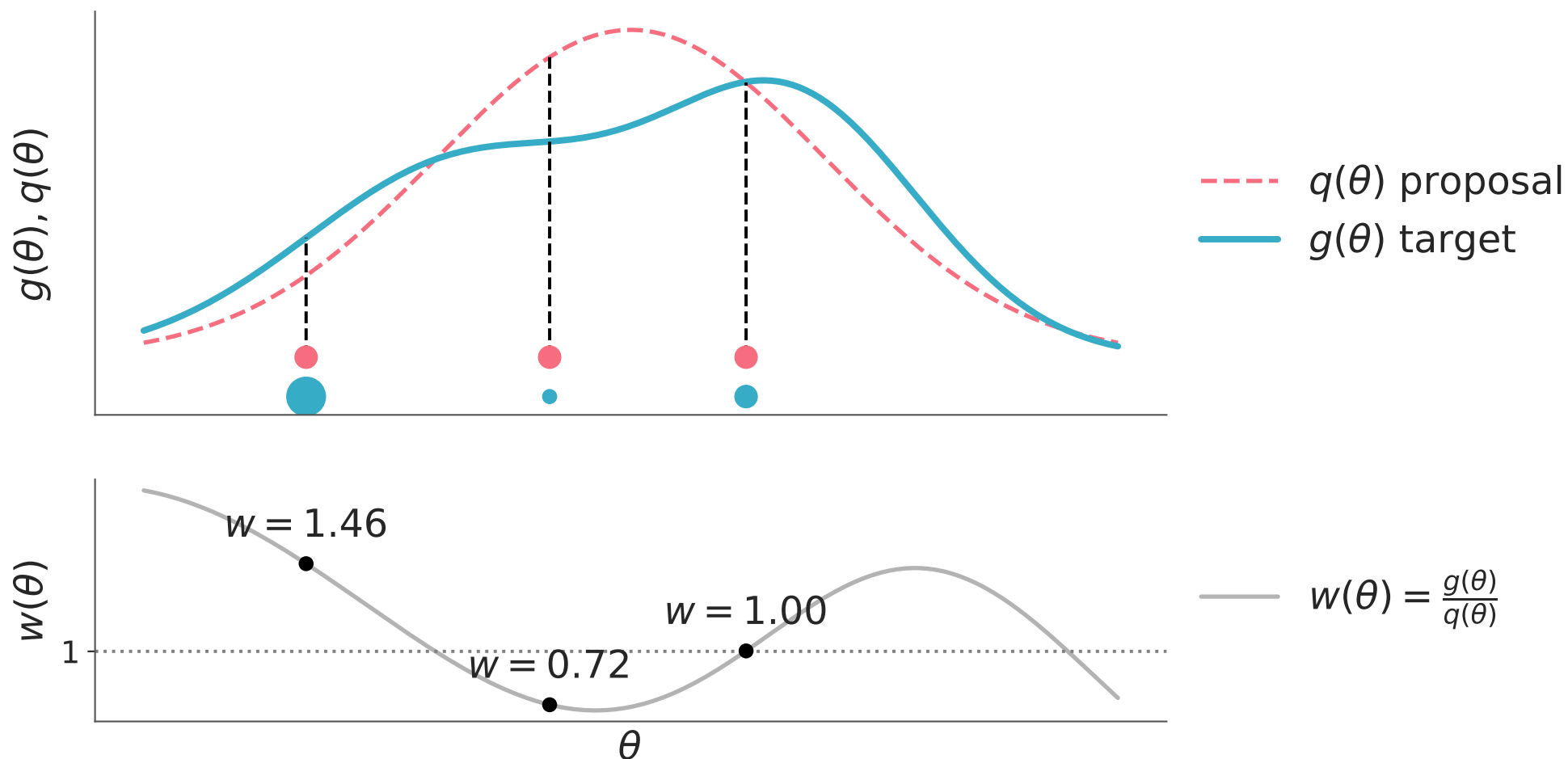


Rejection sampling

- When using direct sampling all proposed draws are accepted
 - The sample size (for MCSE calculation) is the number of accepted draws
- In rejection sampling, the number of accepted draws is typically smaller than the number of trials
- The effective sample size (ESS) is the number of accepted draws
 - with bad proposal distribution may require a lot of trials
 - selection of good proposal gets very difficult when the number of dimensions increase

Importance sampling

- Insight: Proposal does not need to have a higher value everywhere
- We can reweight the samples to account for the difference between proposal and target distribution



Importance sampling: the estimator

- Target expectation of interest: $E_q[h(\theta)]$, for some function $h(\theta)$
 - e.g. $h(\theta) = \theta$ gives the posterior mean
- Draw $\theta^s \sim g(\theta)$ from proposal, define weight $w(\theta^s) = \frac{q(\theta^s)}{g(\theta^s)}$
- Importance sampling estimator:

$$E_q[h(\theta)] \approx \frac{\sum_{s=1}^S w(\theta^s) h(\theta^s)}{\sum_{s=1}^S w(\theta^s)}$$

Importance sampling

- Selection of good proposal gets more difficult when the number of dimensions increase
- There are many special use case which scale well (10k dimensions)

Some uses of importance sampling

- Fast leave-one-out cross-validation (lecture 9)
- Fast prior and likelihood sensitivity analysis (lecture 11)
- Other uses we are not going to discuss
 - Fast bootstrapping
 - Conformal Bayesian computation
 - Particle filtering
 - Improving distributional approximations (e.g Laplace, Pathfinder, VI)

IS finite variance and central limit theorem

- If $h(\theta)w$ and w have finite variance $\rightarrow$ CLT
 - variance goes down as $\frac{1}{S}$
 - Effective sample size (ESS) takes into account the variability in the weights

IS finite variance and central limit theorem

- If $h(\theta)w$ and w have finite variance $\rightarrow$ CLT
 - variance goes down as $\frac{1}{S}$
 - Effective sample size (ESS) takes into account the variability in the weights
- We would like to have finite variance and CLT
 - sometimes these can be guaranteed by construction, e.g., by choosing $g(\theta)$ so that $w(\theta)$ is bounded
 - generally not trivial

IS finite variance and central limit theorem

- If $h(\theta)w$ and w have finite variance $\rightarrow$ CLT
 - variance goes down as $\frac{1}{S}$
 - Effective sample size (ESS) takes into account the variability in the weights
- We would like to have finite variance and CLT
 - sometimes these can be guaranteed by construction, e.g., by choosing $g(\theta)$ so that $w(\theta)$ is bounded
 - generally not trivial
- Pre-asymptotic and asymptotic behavior can be really different!

Importance re-sampling

- Using the weighted draws is good

$$E[h(\theta)] \approx \frac{\sum_s w_s h(\theta^s)}{\sum_s w_s}$$

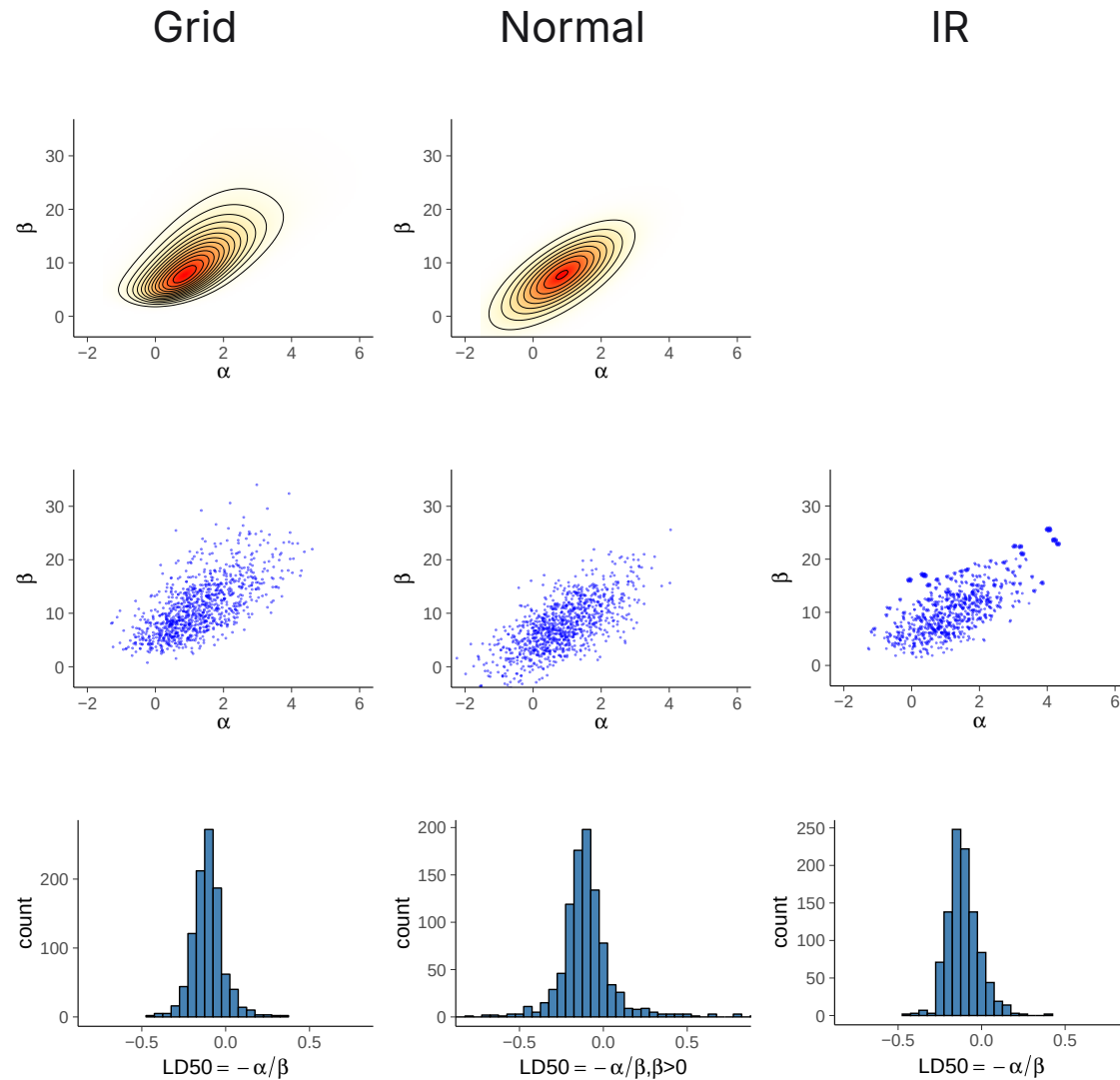
Importance re-sampling

- Using the weighted draws is good

$$E[h(\theta)] \approx \frac{\sum_s w_s h(\theta^s)}{\sum_s w_s}$$

- But it can be convenient to obtain draws with equal weights
 - resample the draws according to the weights
 - some original draws may be included more than once
 - loses some information, but now the weights are equal

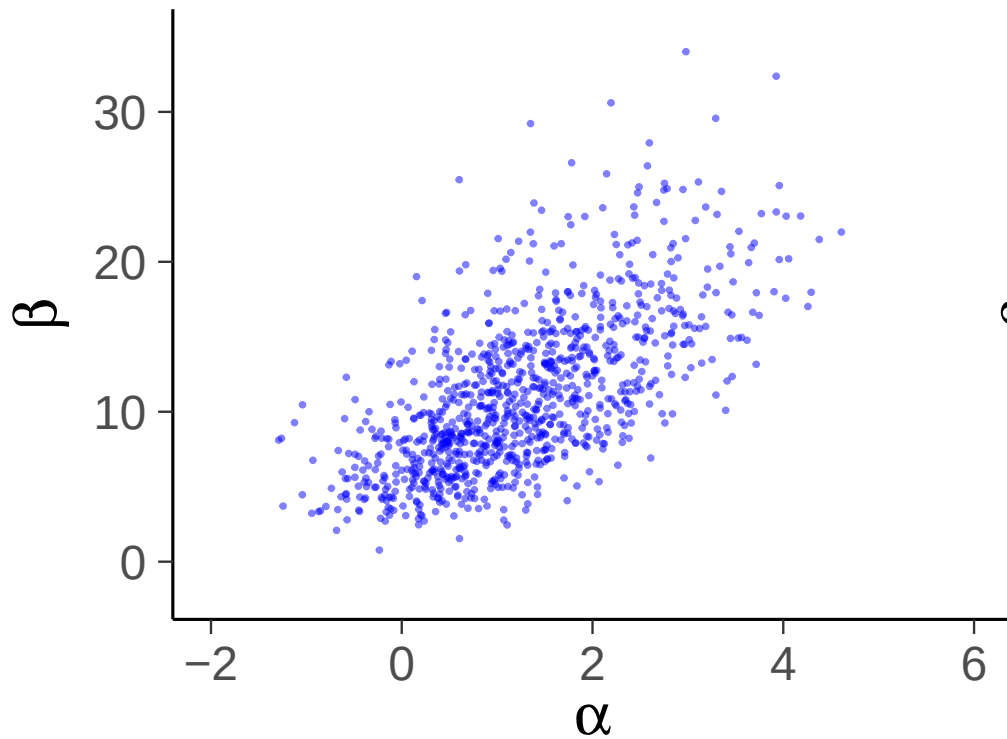
Example: Importance sampling in Bioassay



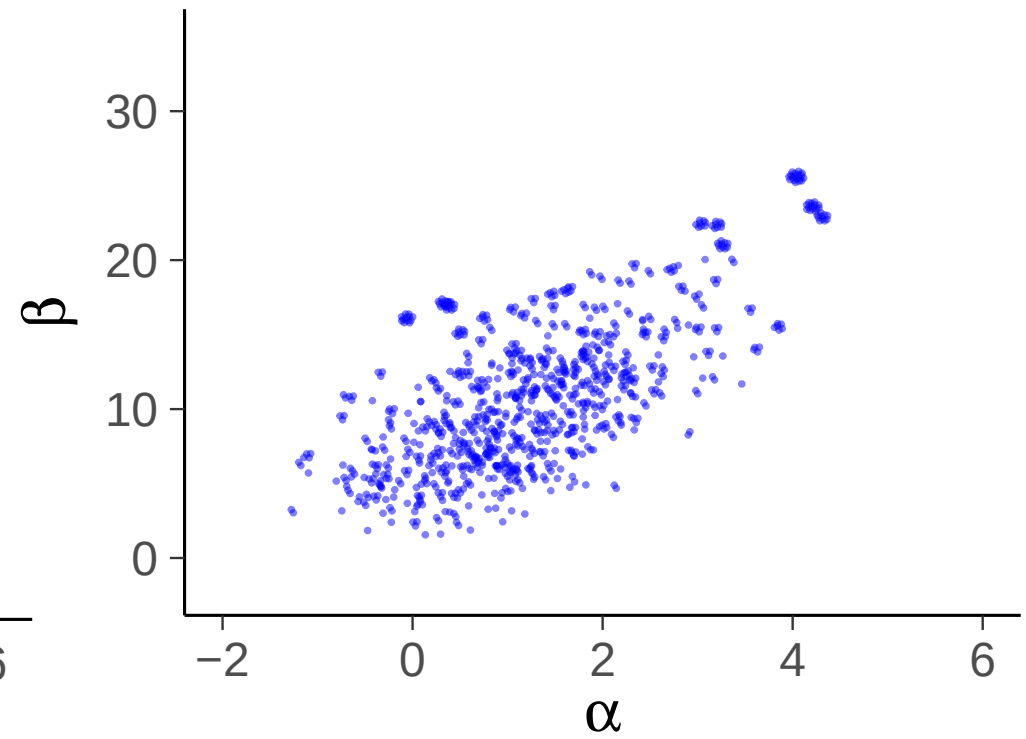
Normal approximation is discussed more in BDA3 Ch 4 But the normal approximation is not that good here: Grid $sd(LD50) \approx 0.1$, Normal $sd(LD50) \approx .75$! IR $sd(LD50) \approx 0.1$

Example: Importance sampling in Bioassay

Grid

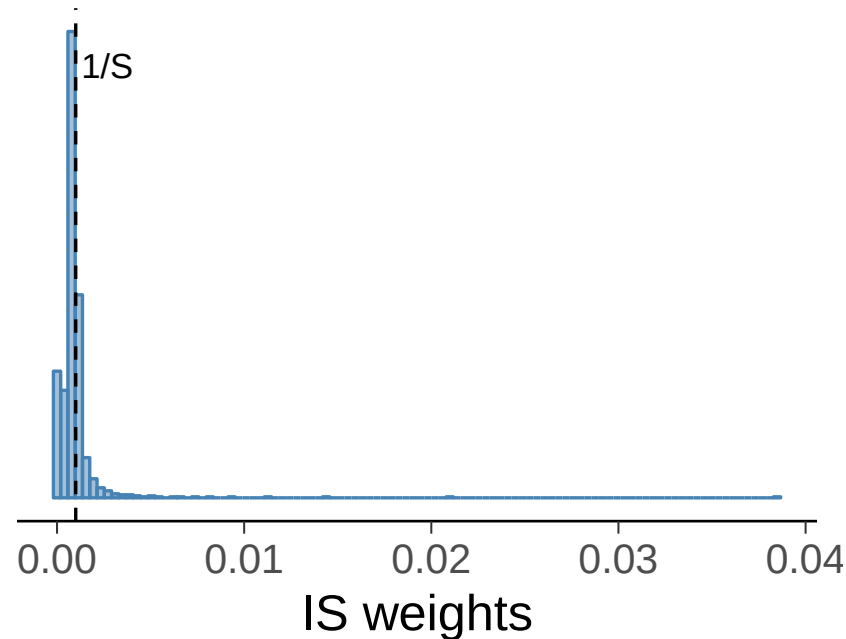


IR



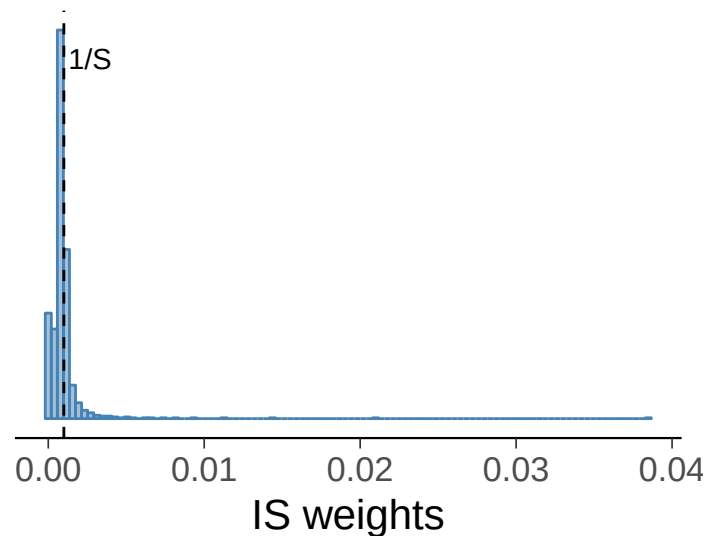
Example: Importance sampling in Bioassay

- Most weights are very small, and a few has very large values
- Larger weights can potentially dominate the estimate



Example: Importance sampling in Bioassay

- We can compute the effective sample size (ESS) to assess the quality of the importance sampling weights



$$\text{ESS} = \frac{1}{\sum_{s=1}^S (\tilde{w}(\theta^s))^2}, \quad \text{where} \quad \tilde{w}(\theta^s) = \frac{w(\theta^s)}{\sum_{s'=1}^S w(\theta^{s'})}$$

$$\text{ESS} \approx 396, \quad S = 1000$$

BDA3 1st (2013) and 2nd (2014) printing have an error for $\tilde{w}(\theta^s)$. The equation should not have the multiplier S (the normalized weights should sum to one). Online version is correct. Errata for the book [errata](#)

Example: Importance sampling in Bioassay

- $S = 1000$
- $\text{ESS} \approx 396$
- If all $\tilde{w}(\theta^s) = \frac{1}{S}$, then $\text{ESS} = \frac{1}{S S^{-2}} = S$
- If one $\tilde{w}(\theta^s) = 1$, and others 0, then $\text{ESS} = \frac{1}{1} = 1$
- Pareto- $\hat{k} \approx 0.65 \rightarrow$ CLT does not hold
- with Pareto-smoothing the estimate would be fine if $\hat{k} < 0.7$

Numerical accuracy – floating point

- Floating point presentation of numbers, e.g. with 64bits
 - closest value to zero is $\approx 2.2 \cdot 10^{-308}$
 - generate sample of 600 from normal distribution: `qr=rnorm(600)`
 - calculate joint density given normal: `prod(dnorm(qr))` → **0 (underflow)**
 - closest value to 1 is $\approx 1 \pm 2.2 \cdot 10^{-16}$
 - Laplace and ratio of girl and boy babies
 - `pbeta(0.5, 241945, 251527)` → **1 (rounding)**
 - `pbeta(0.5, 241945, 251527, lower.tail=FALSE)` $\approx -1.2 \cdot 10^{-42}$ there is more accuracy near 0

Numerical accuracy – log scale

- Log densities
 - use log densities to avoid over- and underflows in floating point presentation
 - `prod(dnorm(qr))` → **0 (underflow)**
 - `sum(dnorm(qr, log=TRUE))` → `-847.3`
 - compute exp as late as possible
 - e.g. for $a > b$, compute $\log(\exp(a) + \exp(b)) = a + \log(1 + \exp(b - a))$
e.g. `log(exp(800)+exp(800))` → **Inf** but `800 + log(1 + exp(800-800))` ≈ 800.69
 - e.g. in Metropolis-algorithm (Assignment 5) compute the log of ratio of densities using the identity $\log\left(\frac{a}{b}\right) = \log(a) - \log(b)$

Numerical accuracy – log scale

- Log densities
 - use log densities to avoid over- and underflows in floating point presentation
 - `prod(dnorm(qr))` → **0 (underflow)**
 - `sum(dnorm(qr, log=TRUE))` → `-847.3`
 - compute exp as late as possible
 - e.g. for $a > b$, compute $\log(\exp(a) + \exp(b)) = a + \log(1 + \exp(b - a))$
e.g. $\log(\exp(800) + \exp(800)) \rightarrow \text{Inf}$ but $800 + \log(1 + \exp(800 - 800)) \approx 800.69$
 - e.g. in Metropolis-algorithm (Assignment 5) compute the log of ratio of densities using the identity $\log\left(\frac{a}{b}\right) = \log(a) - \log(b)$
- convenience functions
 - `matrixStats::logSumExp(lx)` computes $\log(\sum(\exp(lx)))$ using the above rule
 - `log1p(x)` computes $\log(1+x)$ accurately also for $|x| \ll 1$
 - `expm1(x)` computes $\exp(x) - 1$ accurately also for $|x| \ll 1$

Appendix: MCSE for quantiles

- Posterior probability of an event A :

$$p(\theta \in A) \approx \frac{1}{S} \sum_{s=1}^S I(\theta^{(s)} \in A), \quad I(\theta^{(s)} \in A) = \begin{cases} 1 & \text{if } \theta^{(s)} \in A \\ 0 & \text{otherwise} \end{cases}$$

- This is the fraction of draws in A
- Each indicator is a coin flip with probability p , so their sum is $\text{Binomial}(S, p)$
- The variance σ^2 of the Binomial sum is $Sp(1 - p)$
- The MCSE is $\frac{\sigma^2}{\sqrt{S}} \rightarrow \frac{\sqrt{Sp(1-p)}}{S} = \sqrt{\frac{p(1-p)}{S}}$

Example: estimating a small probability

- $S = 1000$ draws, $\hat{p} = 0.05$
- $\text{MCSE} = \sqrt{0.05 \times 0.95 / 1000} \approx 0.0069$
- Normal-approximation 90% interval: $\hat{p} \pm 1.645 \times \text{MCSE} \approx (0.04, 0.06)$
- Normal approximation can fail so using an exact Beta interval instead may be preferable, since normal can go outside $(0, 1)$
- Rule of thumb: estimating a small p reliably needs $S \gg 1/p$ draws, otherwise you may get zero draws in A

Beta interval

- If $x = S\hat{p}$ draws land in A (out of S), the exact sampling interval for p comes from:

$$p \sim \text{Beta}(x + 1, S - x + 1)$$

- Example: $S = 100$, $\hat{p} = 0.05 \rightarrow x = 5$ draws in A

$$p \sim \text{Beta}(6, 96)$$

- 90% interval = 5th and 95th percentiles of this Beta distribution
- In practice: `qbeta(c(0.05, 0.95), 6, 96)` (R) or `beta.ppf([0.05, 0.95], 6, 96)` (scipy.stats, Python)
- Result: (0.02, 0.11) — versus the normal approximation's (0.001, 0.099)
- The Beta interval is narrower on the low side and wider on the high side, it respects the fact that p can't go below 0

From probability uncertainty to quantile uncertainty

- The p -quantile A_p is the cutoff such that a fraction p of θ falls below it:
 $\Pr(\theta < A_p) = p$
- We estimate probabilities by counting draws below a fixed cutoff; we estimate quantiles by finding the cutoff that gives a target count
- Point estimate: sort the S draws, take $A_p = \theta_{(k)}$ at rank $k = p(S + 1)$
- We already have an uncertainty interval for the probability, reuse it to get an interval for A_p :
 - lower bound of p 's interval $\rightarrow$ a rank $\rightarrow$ a draw $= A^-$
 - upper bound of p 's interval $\rightarrow$ a rank $\rightarrow$ a draw $= A^+$
- (A^-, A^+) is the uncertainty interval for the quantile

Example: quantile uncertainty interval

- Continuing $S = 1000$: point estimate $A_{0.05} = \theta_{(50)}$ (rank $0.05 \times 1001 \approx 50$)
- 90% interval for p was $(0.04, 0.06)$ (previous slide)
- Convert each bound to a rank: $k^- = 0.04 \times 1001 \approx 40$, $k^+ = 0.06 \times 1001 \approx 60$
- Read off the draws at those ranks: $A^- = \theta_{(40)}$, $A^+ = \theta_{(60)}$
- $(\theta_{(40)}, \theta_{(60)})$ is our 90% interval for the 5th percentile

Tail quantiles are harder to estimate

- Central quantiles (e.g. the median): draws are dense there, so nearby ranks correspond to nearby θ values $\rightarrow$ narrow interval
- Tail quantiles (e.g. the 1st percentile): draws are sparse there, so the same spread in ranks corresponds to a much wider spread in $\theta \rightarrow$ wide interval
- Same S , same rank-spread, but a much less precise estimate in the tail
- See worked examples in the [digits case study](#)
- This is a simplified version of the idea, see [Vehtari, Gelman, Simpson, Carpenter, & Bürkner \(2021\)](#)